VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BCSE307P – COMPILER DESIGN LAB

PROJECT REPORT



SheetGuard: A Compiler-Based Verification Engine for Spreadsheet Formulas

Submitted by

Harsh Vardhan Golhani – 24BAI0242

Utkarsh Raj – 24BCE2868

Shrijay Sinha – 24BCE2998

Under the guidance of

Nithya K,
Assistant Professor Senior Grade II

OCTOBER, 2026

BONAFIDE CERTIFICATE

This is to certify that the project work titled **“SheetGuard: A Compiler-Based Verification Engine for Spreadsheet Formulas”** is a bonafide record of work carried out by **Harsh Vardhan Golhani (24BAI0242), Utkarsh Raj (24BCE2868) and Shrijay Sinha (24BCE2998)**, submitted in partial fulfilment of the requirements for the course BCSE307P – Compiler Design Lab, School of Computer Science and Engineering, VIT Vellore, during the academic year 2026 - 2027.

Faculty Guide: Nithya K

Date: 28 October 2026

DECLARATION

We hereby declare that the project report titled **"SheetGuard: A Compiler-Based Verification Engine for Spreadsheet Formulas"** submitted by us to VIT Vellore in partial fulfilment of the requirement for the award of the degree/course BCSE307P – Compiler Design Lab is a record of bona fide work carried out by us under the supervision of **Nithya K, Assistant Professor Senior Grade II**. The contents of this report, in full or in part, have not been submitted to any other institute or university for the award of any degree or diploma.

Place: Vellore

Date: 28 October 2026

Signatures of the team members

Harsh Vardhan Golhani (24BAI0242)

Utkarsh Raj (24BCE2868)

Shrijay Sinha (24BCE2998)

ABSTRACT

Spreadsheets are the world's most widely used programming environment, yet their formulas get none of the checks a compiler gives an ordinary program, and audits repeatedly find errors in most of them. **SheetGuard** treats a spreadsheet as source code and runs a complete compiler pipeline over it. A hand-written lexer and a recursive-descent parser turn every formula into an abstract syntax tree; a symbol table resolves sheets, named ranges and 213 built-in functions; a whole-workbook dependency graph, analysed with Tarjan's and Kahn's algorithms, serves as the intermediate representation; a type checker with a unit lattice (currency, percentage, date) and a smell detector based on the R1C1 canonical form find semantic bugs; and an optimizer performs constant folding and dead-cell elimination. Every finding carries one of 23 diagnostic codes, a plain-English explanation and, where possible, an automatic fix that is re-verified by recompiling the corrected formula.

On the synthesis side the same intermediate representation is compiled to five execution engines (an AST interpreter, generated Python, NumPy, JavaScript and SQL) that agree on every cell; compiled and vectorised code evaluates a 10,012-formula model about 720 times faster than naive re-interpretation. A syllabus laboratory derives a conflict-free LL(1) table, a 79-state SLR(1) automaton, three-address code and register-machine code for the same language. SheetGuard is delivered as a Google Sheets add-on, a web application and a command-line tool, finds all 31 bugs planted in two test workbooks, and passes 131 automated tests.

Keywords: spreadsheet verification, lexical analysis, recursive-descent parsing, dependency graph, type and unit inference, static analysis, code generation, LL(1), SLR(1), three-address code.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION..... 6
1.1 Background..... 6
1.2 Problem Statement..... 7
1.3 Objectives & Novelty..... 7
CHAPTER 2 LITERATURE SURVEY..... 9
2.1 Existing Systems / Related Work..... 9
2.2 Research Gap..... 10
CHAPTER 3 SYSTEM DESIGN..... 11
3.1 Proposed System Architecture..... 11
3.2 Module Description..... 12
3.3 Compiler Phases Implementation..... 13
3.4 Workflow..... 19
3.5 Module-wise References..... 21
CHAPTER 4 IMPLEMENTATION DETAILS..... 23
4.1 Implementation Details..... 23
4.2 Screenshots..... 24
4.3 Current Progress and Challenges..... 40
4.4 Conclusion and Future Enhancements..... 42
REFERENCES..... 43

PROJECT SOURCE CODE (GITHUB REPOSITORY)

https://github.com/harshvgolhani/Compiler-Design-Project

CHAPTER 1
INTRODUCTION

1.1 Background

A spreadsheet is a program. Every formula such as `=SUM(Sales!B2:B9)*(1+TaxRate)` is an expression in a small, purely functional language; every cell is a variable; and the workbook as a whole is a data-flow program whose evaluation order is determined by the references between cells. Hundreds of millions of people write programs in this language, most of whom would not call themselves programmers, and organisations use the results to set budgets, prices, salaries, grades and public policy.

Unlike a conventional programming language, however, the spreadsheet language has almost no static checking. A formula that adds a price to a percentage, refers to a mistyped name, sums one row too few or was overwritten by a typed number is accepted silently and simply produces a wrong number. Field audits summarised by Panko [2] found errors in the large majority of operational spreadsheets examined. Several well-known incidents show the cost:

- In the influential *Growth in a Time of Debt* study, a spreadsheet average omitted several countries because its range stopped too early, an off-by-one range error found by Herndon, Ash and Pollin [3].
- JPMorgan's internal task-force report on the 2012 “London Whale” losses describes a value-at-risk model that was operated through copied-and-pasted spreadsheets and in which a calculation divided by a sum instead of an average [4].
- In October 2020 Public Health England failed to report nearly 16,000 positive COVID-19 test results because data passed through a legacy spreadsheet format with a fixed row limit [18].

Each of these is exactly the kind of fault a compiler detects routinely in other languages: a range check, an inconsistent expression across a loop body, or a type mismatch. This observation motivates SheetGuard: if a spreadsheet is a program, then the techniques taught in a compiler design course (lexical analysis, parsing, symbol tables, intermediate representations, semantic analysis, optimisation and code generation) can be applied to it directly, and the resulting tool can explain problems to the people who wrote the formulas.

1.2 Problem Statement

Given a workbook (a set of sheets containing constants and formulas, plus named ranges), the problem is to **automatically detect, explain and, where safe, correct the errors in its formulas**, without executing the user's business process and without requiring the author to change how they work. Concretely, the system must:

- parse every formula of the spreadsheet language exactly as Google Sheets and Excel interpret it, including their quirks (for example, unary minus binds tighter than exponentiation, so `=-2^2` is 4);
- build a representation of the whole workbook that supports reasoning across cells and sheets (cycles, evaluation order, impact of a change);
- find syntax, naming, type, unit, dimension and structural errors, and rank them by severity;
- propose fixes that are guaranteed not to introduce new errors, and show what the errors are costing in the reported figures;
- run where spreadsheet users already work, inside Google Sheets, while remaining usable from a browser and a terminal.

1.3 Objectives & Novelty

Objectives.

- Design a formal grammar for the spreadsheet formula language and implement a hand-written lexer and recursive-descent parser with precise, caret-positioned error messages and phrase-level error recovery.
- Implement a block-structured symbol table (built-in, workbook and sheet scopes), a whole-workbook dependency DAG with cycle detection (Tarjan), topological levels (Kahn) and impact analysis.
- Implement semantic analysis: a type lattice with units (currency, percent, date), dimension checking for lookups and ranges, and division-by-empty detection.
- Detect structural spreadsheet smells (copy-paste inconsistency, off-by-one ranges, magic numbers, overwritten formulas, double counting) with compiler techniques such as the R1C1 canonical form.
- Implement an optimiser (constant folding, algebraic simplification, dead-branch and dead-cell elimination, slicing) and several code generators, and demonstrate the speed-up of compilation over interpretation.

- Give the syllabus techniques a concrete treatment on the same language: FIRST/FOLLOW, LL(1) and SLR(1) parsing tables with traces, three-address code, basic blocks, DAG optimisation and register-machine target code.
- Deliver the system as a Google Sheets add-on, a web application and a command-line tool, and validate it with automated and differential tests.

Novelty. Existing spreadsheet checkers (Section 2.1) are either rule-based per cell or statistical. SheetGuard differs in five ways:

- **A complete classical pipeline.** Every analysis runs on a real AST and dependency graph, and the grammar is proved LL(1) and SLR(1) by construction of conflict-free tables.
- **Units as types.** Unit algebra is integrated into the type checker, so `price + growth%` is a type error rather than a heuristic warning.
- **Verified auto-fixes.** Every automatic fix is re-compiled and re-analysed before it is offered (a lightweight form of translation validation [16]), and its effect on the reported figures is quantified.
- **Five agreeing back-ends.** The same intermediate representation is compiled to an interpreter, Python, NumPy, JavaScript and SQL, and the engines are differentially tested against each other [15].
- **Dynamic analysis.** A type-guided fuzzer finds latent errors that only appear for some inputs, with a witness input and a guard.

CHAPTER 2
LITERATURE SURVEY

2.1 Existing Systems / Related Work

Compiler foundations. The design follows the classical structure described by Aho, Lam, Sethi and Ullman [1]: a scanner and a predictive (recursive-descent) parser for the front end, symbol tables for name resolution, an intermediate representation for analysis and optimisation, and code generation. Strongly connected components are computed with Tarjan's depth-first algorithm [13] and evaluation order with Kahn's topological sort [14]. The formula grammar follows the OpenFormula specification [17] and the observed behaviour of Google Sheets and Excel.

Built-in spreadsheet checks. Excel's background error checking flags individual cells (for example “inconsistent formula”, “formula omits adjacent cells” and “number stored as text”), and both Excel and Google Sheets warn about circular references and offer precedent/dependent tracing. These checks are local rules over one cell at a time; they do not infer types or units, do not reason about the whole workbook, and do not verify the corrections they suggest.

Type and unit inference. Erwig and Burnett [5] introduced unit reasoning for spreadsheets (“adding apples and oranges”), using row and column headers as units. UCheck by Abraham and Erwig [6] turned this into a type checker for end users. SheetGuard adopts the idea of units as types but derives units from number formats and named inputs (currency, percentage, date) and combines them with dimension checking.

Smell detection. Hermans, Pinzger and van Deursen [7] transferred code smells to spreadsheet formulas (for example long formulas and many references). CUSTODES [8] clusters cells by strong and weak features to find cells that deviate from their cluster, and ExceLint [9] detects formulas whose *reference shape* is anomalous compared with nearby formulas. SheetGuard's copy-paste detector uses the same underlying insight (formulas copied across a region should share one relative form) but implements it as a compiler canonicalisation, the R1C1 form, and reports a concrete corrected formula.

Data debugging and execution. CheckCell [10] finds input values that have an unusually large effect on outputs. Sestoft [11] describes how spreadsheets can be compiled rather than interpreted, and Peyton Jones, Blackwell and Burnett [12] treat the formula language as a functional language

that deserves proper abstraction. Differential testing [15] and translation validation [16] are standard techniques for gaining confidence in compilers; SheetGuard applies both to its own engines and fixes.

Table 2.1: Comparison of SheetGuard with related systems

System	Approach	What it does not cover
Excel / Google Sheets checks	Per-cell rules, circular-reference warning, precedent tracing	No types or units, no whole-workbook analysis, fixes not verified, few explanations
UCheck [6]	Unit inference from headers	Units only; no structural smells, fixes or execution
Hermans et al. [7]	Code smells for formulas	Metrics-based; no semantic types, no correction
CUSTODES [8]	Cell clustering, outlier detection	Statistical; no grammar-based analysis or fix validation
ExceLint [9]	Reference-shape anomaly detection	Excel-only, structural only; no units, cycles, names or execution
CheckCell [10]	Input-impact data debugging	Finds suspicious data, not formula errors
Sestoft [11]	Compiling spreadsheets	Focus on performance, not verification
SheetGuard (this work)	Full compiler pipeline, unit-typed, verified fixes, 5 back-ends	Array/dynamic functions are recognised but not executed (Section 4.3)

2.2 Research Gap

The survey shows four gaps that SheetGuard addresses:

- **No single tool applies the whole compiler pipeline.** Existing work focuses on one analysis (units, smells or anomalies). Bugs often need several phases together: a hard-coded `0.18` is only recognisable as a duplicate of `TaxRate` after symbol resolution and constant evaluation.
- **Fixes are rarely offered and never validated.** Detected problems are shown as warnings; the user must work out the correction. SheetGuard proposes concrete formulas and re-compiles each one, rejecting any fix that would introduce a new error.
- **Results are not explained in business terms.** Tools report cell addresses and rule names. SheetGuard evaluates the workbook before and after the fixes and reports which reported figures are wrong and by how much, in plain English.
- **Little support for Google Sheets.** Most academic tools target Excel files offline. SheetGuard runs as a Google Sheets add-on that reads the live sheet, highlights problem cells and writes fixes back, while sharing one code base with a web application and a command-line interface.

CHAPTER 3
SYSTEM DESIGN

3.1 Proposed System Architecture

Figure 3.1 shows the overall architecture. Three front ends (the Google Sheets add-on, the web application and the command line) convert their input into one **workbook model**: sheets of cells, each holding the raw text the user typed, its cached value and its number format, plus the named ranges. The Sheets add-on and the web application talk to the Python back-end through a small JSON-over-HTTP API, so all analysis runs in one place.

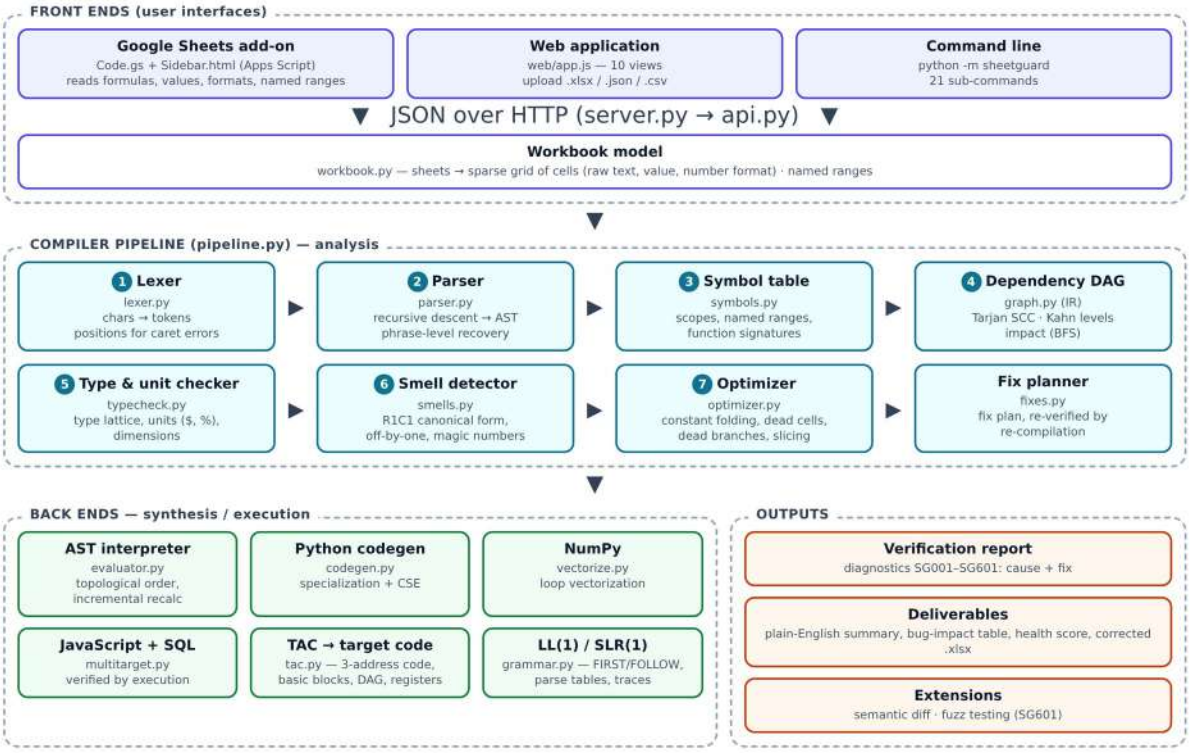


Figure 3.1: Overall architecture of SheetGuard: front ends, compiler pipeline, back ends and outputs

The **compiler pipeline** (`pipeline.py`) runs seven analysis phases in order, each consuming the output of the previous one. The lexer and parser produce an AST for every formula; the symbol table resolves names; the dependency DAG links the cells into one program graph; the type checker and smell detector add semantic findings; and the optimiser simplifies expressions and finds dead cells. The fix planner merges the findings, plans corrections and verifies them. The **back ends** reuse the same ASTs and DAG for execution (interpreter, compiled Python, NumPy,

JavaScript, SQL) and for the syllabus artefacts (parse tables, three-address code, target code). The **outputs** are the verification report, the deliverables (summary, impact table, health score, corrected workbook) and the extensions (semantic diff and fuzz testing).

3.2 Module Description

The implementation consists of about 11,000 lines of Python in 38 modules (no third-party dependency is required for the core), about 1,700 lines of HTML, CSS and JavaScript for the shared user interface and about 300 lines of Google Apps Script. Table 3.1 lists the modules by phase.

Table 3.1: Modules of SheetGuard

Phase / role	Module(s)	Responsibility
Lexical analysis	`tokens.py`, `lexer.py`	Hand-written scanner with one-character lookahead; 18 token types; caret-positioned lexical errors
Syntax analysis	`parser.py`, `ast_nodes.py`	Recursive-descent parser (one function per grammar rule) building 13 kinds of AST node; phrase-level error recovery
Symbol table	`symbols.py`, `functions.py`	Nested scopes, named ranges, cross-sheet link map, catalogue of 213 function signatures (arity, argument kinds, result type)
Intermediate representation	`graph.py`	Whole-workbook dependency DAG; Tarjan SCC, Kahn levels, impact analysis
Semantic analysis	`typecheck.py`	Type lattice with units and ranges; unit algebra; dimension and division checks
Static analysis	`smells.py`, `transforms.py`	R1C1 canonical form; copy-paste, off-by-one, magic-number, overwrite and double-counting detectors
Optimisation	`optimizer.py`	Constant folding, identities, dead branches, constant propagation, dead cells, slicing
Error handling	`errors.py`, `diagnostics.py`, `fixes.py`	Diagnostic records (23 codes), fix planning, fix verification, annotated .xlsx export
Execution	`runtime.py`, `evaluator.py`	Spreadsheet value semantics for 129 executed functions; AST interpreter; incremental recalculation
Code generation	`codegen.py`, `vectorize.py`, `multitarget.py`	Straight-line Python with type specialisation and CSE; NumPy vectorisation; JavaScript and SQL back-ends
Syllabus artefacts	`grammar.py`, `tac.py`	FIRST/FOLLOW, LL(1) table and parser, LR(0)/SLR(1) automaton and parser; TAC, basic blocks, DAGs, register-machine code
Extensions	`diff.py`, `fuzz.py`, `summary.py`	Semantic diff of two versions; type-guided fuzzing; plain-English summary, impact report, health score
Interfaces	`api.py`, `server.py`, `__main__.py`, `web/`, `addon/`	JSON API and HTTP server; 20 CLI commands; web UI with 10 views; Google Sheets sidebar and graph dialog

Phase / role	Module(s)	Responsibility
Support	`workbook.py`, `doctor.py`, `stack.py`	Loaders for .xlsx/.csv/.json; installation self-test; large-stack threads for deeply nested formulas

3.3 Compiler Phases Implementation

This section explains how each compiler phase is implemented, using the demonstration workbook *Acme Retail – Q3 Sales Report* (three sheets, 59 formulas, 14 planted bugs) as the running example.

3.3.1 Phase 1: Lexical Analysis

The lexer (`lexer.py`) is a hand-written scanner: a deterministic finite automaton expressed as code. It reads the formula one character at a time and groups characters into tokens using the longest-match rule. Each token records its type, its lexeme and its start and end positions in the original text, so that every later error can point to the exact column.

The interesting part is classifying identifiers, because the same characters mean different things depending on what follows them. One character of lookahead resolves this:

Table 3.2: Classification of lexemes

Input	Token	Rule
`SUM(`	FUNC	identifier followed by `(`
`LOG10`	CELL	letters + digits form a valid cell address (column LOG, row 10)
`Sales!`, `'Q3 Data'!`	SHEET	identifier or quoted name followed by `!`
`A:C`, `2:5`	COL_RANGE, ROW_RANGE	whole-column / whole-row ranges
`TRUE`, `FALSE`	BOOL	keywords
`#DIV/0!`, `#N/A`	ERROR	error literals
`TaxRate`	NAME	any other identifier: a defined name, resolved by the symbol table
`12.5`, `1E-3`, `"text"`	NUMBER, STRING	numeric and string literals (`""` escapes a quote)

The remaining token types are OP (`+ - * / ^ & = <> < > <= >= %`), COLON, COMMA, SEMI, the brackets `( ) { }` and EOF. Lexical errors such as an unterminated string literal or a malformed number (`12abc`) are reported with a caret under the offending characters (Figure 4.23). The token stream for the running example is:

```
=SUM(Sales!B2:B9)*(1+TaxRate)

FUNC(SUM) LPAREN SHEET(Sales) CELL(B2) COLON CELL(B9) RPAREN
OP(*) LPAREN NUMBER(1) OP(+) NAME(TaxRate) RPAREN EOF
```

3.3.2 Phase 2: Syntax Analysis

The parser (`parser.py`) is a hand-written **recursive-descent** parser. The grammar has one non-terminal per precedence level, lowest precedence first, and the parser has exactly one function per non-terminal. Left-associative operators are parsed with loops, which is the standard way to remove left recursion in a top-down parser:

```
formula     ::= '=' expr EOF
expr        ::= comparison
comparison  ::= concat   ( ('=' | '<>' | '<' | '>' | '<=' | '>=') concat )*
concat      ::= additive ( '&' additive )*
additive    ::= term     ( ('+' | '-') term )*
term        ::= power    ( ('*' | '/') power )*
power       ::= unary    ( '^' unary )*            -- left-associative, like Excel
unary       ::= ('-' | '+') unary | postfix        -- binds tighter than '^'
postfix     ::= primary '%'*
primary     ::= NUMBER | STRING | BOOL | ERROR | reference | NAME
              | FUNC '(' [ args ] ')' | '(' expr ')' | '{' array '}'
reference   ::= [SHEET] CELL [ ':' [SHEET] CELL ]
              | [SHEET] COL_RANGE | [SHEET] ROW_RANGE
args        ::= arg ( (',' | ';') arg )*           -- an argument may be empty
```

Two quirks of the spreadsheet language are encoded in the grammar: exponentiation is left-associative (`2^3^2` is 64), and unary minus binds tighter than exponentiation (`-2^2` is 4, not −4). The parser produces an abstract syntax tree of 13 node types (`Number`, `String`, `Bool`, `ErrorLit`, `Missing`, `ArrayLit`, `CellRef`, `RangeRef`, `NameRef`, `BinOp`, `UnaryOp`, `Percent`, `FuncCall`). Every node can be printed back to formula text (`unparse`), and tests check that parsing and printing round-trip. Nesting is limited to 100 levels to protect the stack.

Error recovery. When parsing fails, the parser performs **phrase-level recovery**: it tries the smallest local edits around the error position (insert a missing `)`, delete the offending token, close an unterminated string before the trailing parentheses, drop a dangling operator) and returns the first candidate that parses. For `=SUM(Sales!D2:D9` the report says “missing ')' to close SUM(” and suggests `=SUM(Sales!D2:D9)`.

3.3.3 Phase 3: Symbol Table and Name Resolution

Names in a workbook are scoped like a block-structured language. The symbol table (`symbols.py`) has three nested scopes: the **built-in** scope holds the 213 function signatures

(minimum and maximum arity, argument kinds, result type); the **workbook** scope holds sheet names and workbook-level named ranges; and each **sheet** scope holds sheet-local names that shadow workbook names. Resolution walks from the innermost scope outwards.

Cross-sheet references such as `Rates!B2` behave like external symbols in separate object files: the resolver checks that the target sheet exists and records the link, which produces a *link map* showing which sheet imports from which. This phase reports unknown functions (SG101, with "did you mean" suggestions ranked by edit distance, for example `SUMM` → `SUM`), wrong argument counts (SG102), undefined names (SG103, `TaxRte` → `TaxRate`), missing sheets (SG104), and functions that are recognised but not executed (SG106).

3.3.4 Phase 4: Intermediate Representation – the Dependency DAG

The second intermediate representation is a whole-workbook **dependency graph** (`graph.py`). Nodes are cells; an edge P → D means that D's formula reads P. Ranges are expanded over the non-empty cells they cover, with whole-column ranges clipped to the used area. Three classical algorithms run on this graph:

- **Tarjan's strongly-connected-components algorithm** [13] finds circular references in O(V + E). In the demo, Summary!B4 (bonus = 10% of profit) and Summary!B5 (profit = revenue − commission − bonus) form a two-cell cycle (SG201).
- **Kahn's topological sort** [14], computed level by level, gives the minimal correct recalculation order. All cells on one level are independent and could be computed in parallel; the demo workbook has 8 levels. Cells in a cycle, and cells that depend on them, are reported as blocked.
- **Forward and backward reachability (BFS)** answers impact questions: changing Inputs!B3 (Discount) affects 46 cells. The same reachability drives incremental recalculation, where only the affected cells are recomputed.

3.3.5 Phase 5: Semantic Analysis – Types, Units and Dimensions

Spreadsheets are dynamically and silently typed. The type checker (`typecheck.py`) infers a static type for every cell over the lattice below and checks every operator and function call against typing rules, like the semantic phase of any compiler. `join` is the least upper bound, and `Any` is the top element used when a value is too dynamic to type.

```
Number[unit]   unit ∈ { plain, currency, percent }
Date           Text (flag: looks numeric)   Bool   Error   Empty
Range<T> rows×cols      (T = join of its cells)       Any = T

currency ± currency → currency        currency ± percent  → ✗ SG302
currency × percent  → currency        currency × currency → ✗ SG302 (money²)
currency ÷ currency → plain (ratio)   date − date         → plain (days)
```

Units come from number formats (`$#,##0.00`, `0.0%`) and from the named inputs a formula reads. The same phase checks **dimensions** (VLOOKUP asking for column 3 of a 2-column table, SUMPRODUCT over ranges of different shapes: SG303), **division by zero or by an empty cell** (SG304), **text used as a number** (SG301) and **numbers stored as text**, such as `"$45.00"`, which SUM silently skips (SG305). In the demo, `Sales!I4 = G4+H4` adds a currency total to a growth percentage and is reported as a unit mismatch.

3.3.6 Phase 6: Static Analysis – Spreadsheet Smells

The smell detector (`smells.py`) encodes the classic spreadsheet-error taxonomy as compiler analyses over the AST and the DAG. Its key tool is the **R1C1 canonical form**: every reference is rewritten relative to the cell that contains it, so correctly copied formulas become textually identical. For example, D2 = `B2*C2` and D3 = `B3*C3` both become `RC[-2]*RC[-1]`, while the buggy D6 = `B6*C5` becomes `RC[-2]*R[-1]C[-1]`.

Table 3.3: Smells detected by static analysis

Code	Smell	Detection technique
SG401	Inconsistent (copy-paste) formula	Cell whose R1C1 form differs from the majority of its row or column run; the majority form is the fix
SG402	Off-by-one range	Aggregate range that stops one cell before (or after) the contiguous data block
SG403	Hard-coded constant	Literal that equals a named input (0.18 = TaxRate) or a business constant
SG404	Formula overwritten by a value	Typed number inside a run of identical R1C1 formulas
SG405	Double counting	Aggregate whose range includes its own subtotal
SG406	Absolute/relative mix-up	Reference that should be anchored with `$` but drifts in copies
SG105	Reference to an empty cell	Formula reads a blank cell in the DAG

3.3.7 Phase 7: Code Optimisation

The optimiser (`optimizer.py`) runs classical passes over the AST and the DAG: **constant folding** (`=12*25000*(1+10%)` → `330000`, SG501), **algebraic simplification** (`x*1`, `x+0`,

applied only when `x` is proven numeric so that text and blank semantics are preserved), **dead-branch elimination** (`IF(TRUE,a,b)` → `a`), **constant propagation** of named constants, **dead-cell elimination** (inputs that nothing reads, such as an "Old FX rate" left behind: SG502) and **program slicing** (the minimal set of cells needed for chosen outputs). Volatile functions (`RAND`, `NOW`, `TODAY`) are never folded, and an expression whose folding would produce an error is left unchanged so the error still appears at run time.

3.3.8 Error Reporting and Verified Auto-Correction

Every finding becomes a `Diagnostic` record with a code, severity (error, warning, note), cell, one-line message, explanation, fix description and, where possible, a suggested formula. Several detectors can propose the same edit for one cell, so `fixes.py` merges them into the single most specific diagnostic. Each candidate fix is then **re-compiled and re-analysed** in the context of the whole workbook, and is offered only if it removes the finding without introducing a new error. The verified fixes can be applied in the sheet, or exported as an annotated `.xlsx` in which fixed cells are green, every change has a comment and the first sheet is the verification report. On the demo workbook, 10 automatic fixes reduce 7 errors and 9 warnings to 3 errors; the remaining three (the circular reference and the division by an empty cell) need a human decision.

3.3.9 Intermediate Code Generation – Three-Address Code

For the syllabus treatment, `tac.py` translates each formula's AST into **three-address code** by syntax-directed translation, shown as quadruples, triples and indirect triples. `IF` is translated with conditional jumps. The leaders algorithm splits the code into **basic blocks** and builds the flow graph, and each block is converted to a **DAG** by value numbering, which drives local optimisations: constant folding, algebraic identities, common-subexpression elimination, copy propagation and dead-code elimination. For `=IF(B2>0, B2*C2*(1+10%)+B2*C2, 0)` the optimiser folds `1+10%` into `1.1` and computes `B2*C2` only once:

```
Three-address code                  Optimized TAC
t1 = B2 > 0                           t1 = B2 > 0
ifFalse t1 goto L1                    ifFalse t1 goto L1
t3 = B2 * C2                          t3 = B2 * C2
t4 = 10 / 100                         t6 = t3 * 1.1
t5 = 1 + t4                           t8 = t6 + t3
t6 = t3 * t5                          t2 = t8
t7 = B2 * C2                          goto L2
t8 = t6 + t7                        L1:
t2 = t8                               t2 = 0
```

```
goto L2                                L2:
L1: t2 = 0                               result = t2
L2: result = t2

folding 10/100 → 0.1, 1+0.1 → 1.1;  CSE: t7 = B2*C2 reuses t3;
copy propagation; dead code t4, t5, t7 removed
```

With two registers, the simple code generator emits 18 instructions (8 loads, 4 stores, 1 spill) for the optimised code, for example `LD R0, B2`, `LD R1, C2`, `MUL R0, R0, R1`.

3.3.10 Code Generation and Execution Engines

The same intermediate representation is compiled to several targets, all sharing the value semantics in `runtime.py`:

- **AST interpreter** (`evaluator.py`): walks the cached ASTs in topological order; cells in a cycle evaluate to `#CIRC!`, which propagates downstream; supports incremental recalculation.
- **Python code generator** (`codegen.py`): compiles the whole workbook into one straight-line Python function. Where the type checker proved both operands numeric, `B2*C2` compiles to a native multiply; this fast path is speculative and *deoptimises* to the always-correct slow path if an error or blank appears at run time. Common subexpressions such as `(1-Discount)` are computed once.
- **NumPy vectoriser** (`vectorize.py`): treats a column of copied formulas as a loop. After a loop-carried-dependence check (running totals stay scalar), the column becomes one array operation.
- **JavaScript and SQL** (`multitarget.py`): JavaScript with a small runtime prelude, and SQL in which the whole workbook is one query with one common table expression per topological level. Both are verified by executing them (Node.js, SQLite) and comparing every cell with the interpreter.
- **Register-machine target code** (`tac.py`): a simple code generator with register and address descriptors, `getReg` and spilling, for a machine with 2 to 8 registers.

3.3.11 Formal Parser Construction: LL(1) and SLR(1)

The production parser is hand-written, so `grammar.py` gives the same language a formal treatment and builds two more parsers mechanically. For the **LL(1)** parser the grammar is transformed (left recursion removed, left-factored) into 25 non-terminals; FIRST and FOLLOW sets are computed by fixed-point iteration and the predictive table M[A, a] has 271 entries over 31

terminals with **0 conflicts**, which proves that the grammar is LL(1). For the **SLR(1)** parser the natural left-recursive grammar is used: CLOSURE and GOTO build the canonical collection of **79 LR(0) item sets**, and FOLLOW sets resolve the reductions, again with **0 conflicts**. Both parsers print their stack traces (Figures 4.13 and 4.14), and tests check that the recursive-descent, LL(1) and SLR(1) parsers accept and reject exactly the same formulas. Two checks are deliberately left to semantic analysis because they are not context-free: rows of an array constant must have equal length, and both ends of `Sheet1!A1:Sheet1!B2` must name the same sheet.

3.4 Workflow

Figure 3.2 shows the end-to-end workflow of SheetGuard, from the moment a user submits a workbook to the moment the verified corrections are applied. The same sequence is followed whichever front end is used; only the first step (how the workbook is submitted) and the last step (how the results are shown) depend on the interface.

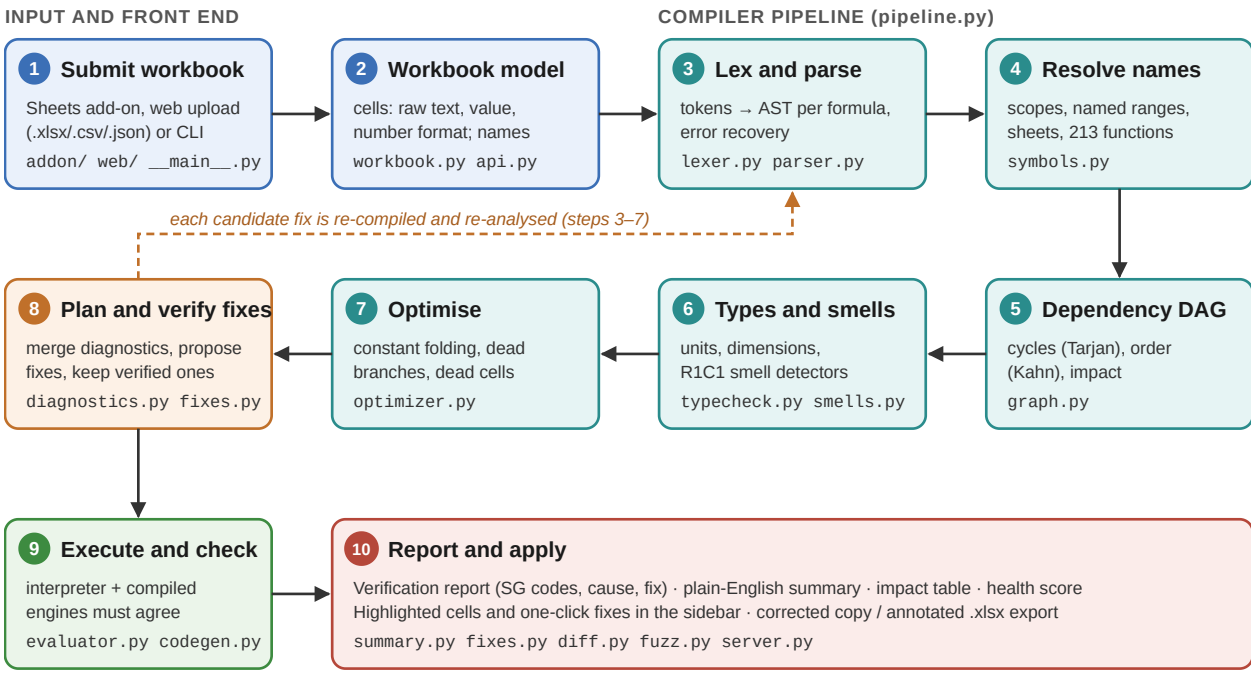


Figure 3.2: Workflow of SheetGuard, from submitting a workbook to applying verified fixes

The steps of the workflow are:

1. **Submit the workbook.** In Google Sheets the sidebar reads the formulas, values, number formats and named ranges of the live spreadsheet; the web application accepts an uploaded `.xlsx`, `.csv` or `.json` file; the command line takes a file path, for example `python -m sheetguard analyze my.xlsx`.
2. **Build the workbook model.** The loaders (`workbook.py`), or the JSON API (`api.py`, `server.py`) for the add-on and the web application, convert the input into one workbook model: sheets of cells, each with its raw text, cached value and number format, plus the named ranges. From this point all front ends share the same pipeline (`pipeline.py`).
3. **Lex and parse every formula.** The lexer turns each formula into tokens and the recursive-descent parser builds its AST. A syntax error is reported with a caret under the exact column and repaired by phrase-level recovery, so analysis of the rest of the workbook continues.
4. **Resolve names.** The symbol table resolves functions, named ranges and sheet references through its nested scopes and records the cross-sheet link map; unknown functions, wrong argument counts, undefined names and missing sheets are reported here (SG101–SG106).
5. **Build the dependency graph.** All cells are linked into one DAG. Tarjan's algorithm finds circular references (SG201), Kahn's algorithm gives the recalculation order level by level, and reachability gives the impact of every input.
6. **Find semantic and structural bugs.** The type checker infers a type and unit for every cell and reports unit mismatches, dimension errors and divisions by zero or by empty cells (SG3xx); the smell detector compares R1C1 forms to find copy-paste, off-by-one, hard-coded-constant, overwrite and double-counting errors (SG4xx).
7. **Optimise.** The optimiser folds constants, simplifies identities, removes dead branches and finds dead cells (SG5xx).
8. **Plan and verify fixes.** Findings from all phases are merged so that each cell keeps its most specific diagnostic. For every candidate fix the edited workbook is re-compiled and re-analysed (steps 3–7 again); a fix is offered only if it removes the finding without introducing a new error.

9. **Execute and cross-check.** The interpreter evaluates the cells in topological order. The same ASTs and DAG are compiled to Python, NumPy, JavaScript and SQL, and the compiled engines are checked against the interpreter.
10. **Report and apply.** The user receives the verification report (code, severity, cell, explanation and suggested formula for every finding), a plain-English summary, the impact table and the health score. In Google Sheets the affected cells are highlighted, a click jumps to a cell, and verified fixes can be applied or a corrected copy created; alternatively an annotated `.xlsx` is exported in which fixed cells are green and every change carries a comment.

On the demonstration workbook (three sheets, 59 formulas, 14 planted bugs) one pass of this workflow reports 7 errors and 9 warnings; 10 verified automatic fixes reduce them to 3 errors (the circular reference and the division by an empty cell), which are left for a human decision.

3.5 Module-wise References

Table 3.4 maps each module of SheetGuard (Table 3.1) to the techniques it implements and the references on which those techniques are based. The numbers refer to the list of references at the end of this report.

Table 3.4: Module-wise references

Module(s)	Technique / concept used	References
`tokens.py`, `lexer.py`	Lexical analysis as a deterministic finite automaton, longest-match rule, lexical error reporting; token set of the formula language	[1], [17]
`parser.py`, `ast_nodes.py`	Recursive-descent (top-down) parsing, removal of left recursion, operator precedence and associativity, phrase-level error recovery; formula grammar	[1], [17]
`symbols.py`, `functions.py`	Block-structured symbol tables with nested scopes; built-in function signatures; spreadsheet functions treated as a functional language	[1], [12], [17]
`graph.py`	Strongly connected components for circular references; topological sort for recalculation order; dependency graphs and input-impact analysis	[13], [14], [11], [10]
`typecheck.py`	Type checking over a lattice; units of measure as types; unit inference for spreadsheets	[1], [5], [6]

Module(s)	Technique / concept used	References
`smells.py`, `transforms.py`	Code smells in spreadsheet formulas; outlier cells by clustering; reference-shape anomalies using the R1C1 relative form	[7], [8], [9], [11]
`optimizer.py`	Constant folding, algebraic simplification, dead-code elimination, constant propagation, program slicing	[1]
`errors.py`, `diagnostics.py`, `fixes.py`	Diagnostic records and error reporting; verifying each fix by re-compilation and re-analysis (translation validation)	[1], [16]
`runtime.py`, `evaluator.py`	Spreadsheet value semantics, AST interpretation, incremental recalculation	[11], [17]
`codegen.py`, `vectorize.py`, `multitarget.py`	Compiling spreadsheets instead of interpreting them; common-subexpression elimination; multiple back ends checked by differential testing	[1], [11], [15]
`grammar.py`, `tac.py`	FIRST/FOLLOW sets, LL(1) table, LR(0) items and SLR(1) parsing; three-address code, basic blocks, DAG of a block, simple code generator	[1]
`diff.py`, `fuzz.py`, `summary.py`	Semantic diff of two versions; type-guided fuzzing and differential testing; impact report	[15], [10]
`api.py`, `server.py`, `__main__.py`, `web/`, `addon/`	Google Sheets add-on sidebar and dialogs served through Apps Script HtmlService; JSON API and command line	[19]
`workbook.py`, `doctor.py`, `stack.py`	Loading workbooks according to the OpenFormula model; self-test checking that every engine agrees with the interpreter	[17], [15]
Test suite (131 tests)	Differential testing of the five engines; validation of automatic fixes	[15], [16]

References [2], [3], [4] and [18] document real spreadsheet failures. They motivate the problem (Section 1.1) rather than the design of a particular module, and are therefore not listed in Table 3.4.

CHAPTER 4
IMPLEMENTATION DETAILS

4.1 Implementation Details

Technology. The compiler is written in Python (3.8 or later) using only the standard library; `openpyxl` is optional for `.xlsx` files and `numpy` for the vectorising back-end. The HTTP server is the standard-library `ThreadingHTTPServer`. The user interface is written in plain HTML, CSS and JavaScript with no framework, so the same code runs as the standalone web application, the Google Sheets sidebar and the full-screen graph dialog. The add-on is written in Google Apps Script (V8 runtime).

Table 4.1: Technology stack and size

Component	Technology	Size
Compiler and engines	Python 3.8+ (standard library)	≈ 11,000 lines, 38 modules
User interface	HTML, CSS, JavaScript (no framework)	≈ 1,700 lines
Google Sheets add-on	Apps Script (V8), HtmlService	≈ 300 lines + generated sidebar
Tests	Python `unittest`	131 tests, ≈ 1,400 lines
Optional	openpyxl, NumPy, Node.js, SQLite	detected at run time by `doctor`

Running SheetGuard. On Windows the user double-clicks `START_SheetGuard.bat`, which locates a stable Python installation, installs the optional packages if missing and opens the web application at `http://localhost:8765`. `CHECK_SETUP.bat` runs the self-test (`python -m sheetguard doctor`) that checks every engine. For the Google Sheets add-on, `Code.gs`, `Sidebar.html` and `Graph.html` are pasted into *Extensions → Apps Script* of a spreadsheet, and `START_TUNNEL.bat` exposes the local server through a Cloudflare quick tunnel whose URL is entered once in the sidebar. The sidebar reads the formulas, values, number formats and named ranges of the live spreadsheet, sends them to the compiler, highlights problem cells in the sheet, jumps to cells, applies fixes and can create a corrected copy.

Command-line interface. Twenty sub-commands expose every phase, for example:

```
python -m sheetguard lex   "=SUM(Sales!B2:B9)*(1+TaxRate)"  # tokens
python -m sheetguard parse "=SUM(Sales!B2:B9)*(1+TaxRate)"  # AST
python -m sheetguard analyze my.xlsx                        # full report
python -m sheetguard fix my.xlsx                            # corrected copy
python -m sheetguard grammar | ll1 "=1+2*3" | slr "=1+2*3"  # parsers
```

```
python -m sheetguard tac "=IF(A1>0,A1*2,0)" --regs 2        # TAC, asm
python -m sheetguard js --verify | sql --verify             # targets
python -m sheetguard bench | fuzz | diff | doctor           # tools
```

Testing. The 131 automated tests cover every phase. Beyond unit tests, the system was hardened by **differential testing** [15]: about 10,000 randomly generated workbooks (garbage formulas, well-formed models and vectorisable columns) were evaluated by all five engines, and every formula cell was compared with the interpreter. Each disagreement found this way (for example, `SUM` over a reference must skip text but `SUM` of a literal must not, and `COUNT` must ignore errors) was fixed and has a regression test. Further cross-checks confirm that the three parsers accept the same language, that three-address code before and after optimisation and the register-machine code (run on a simulator with 2 to 8 registers) compute the same values as the interpreter, and that incremental recalculation equals full recalculation. Two test workbooks with 31 planted bugs in total are analysed end-to-end: every planted bug is found.

Table 4.2: Benchmark on a synthetic 2,000-row model (10,012 formulas, 18,040 cells)

Engine	Time (ms)	Speed-up	Identical results
Naive: re-parse and interpret every formula	462.8	1.0×	✓
AST interpreter (parse once)	163.0	2.8×	✓
Compiled Python, no optimisation	14.6	31.8×	✓
+ type specialisation, CSE, folding	3.4	137.6×	✓
+ NumPy loop vectorisation	0.64	720.9×	✓

Incremental recalculation uses the DAG to recompute only affected cells: changing one row's units recomputes 17 of the 10,012 formulas. Timings vary between runs; every engine produces identical values, which the benchmark checks on every cell.

4.2 Screenshots

The following screenshots were captured from the running application (web application at `localhost`, Google Sheets sidebar interface and command line). They show successful compilation, error detection, automatic correction and the syllabus artefacts.

4.2.1 Web application

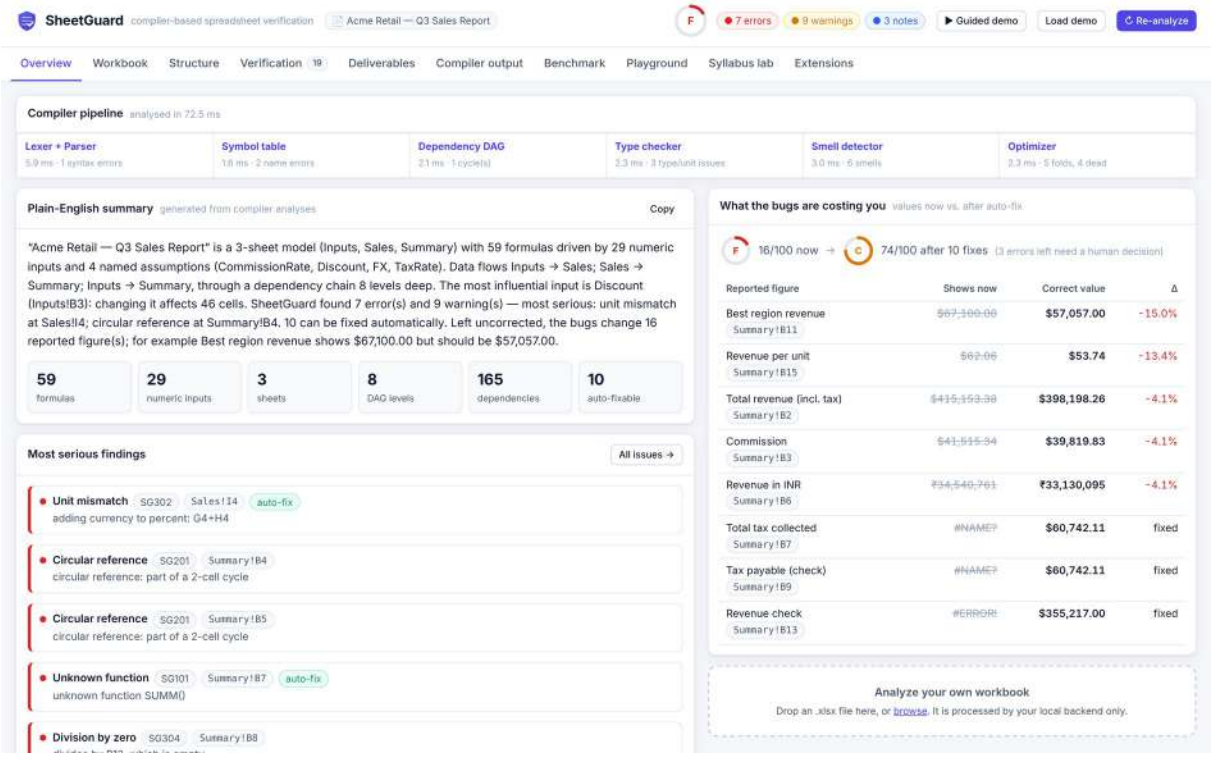


Figure 4.1: Overview: the compiler pipeline with per-phase timings and findings, the generated plain-English summary and the bug-impact table (values now vs. after auto-fix)

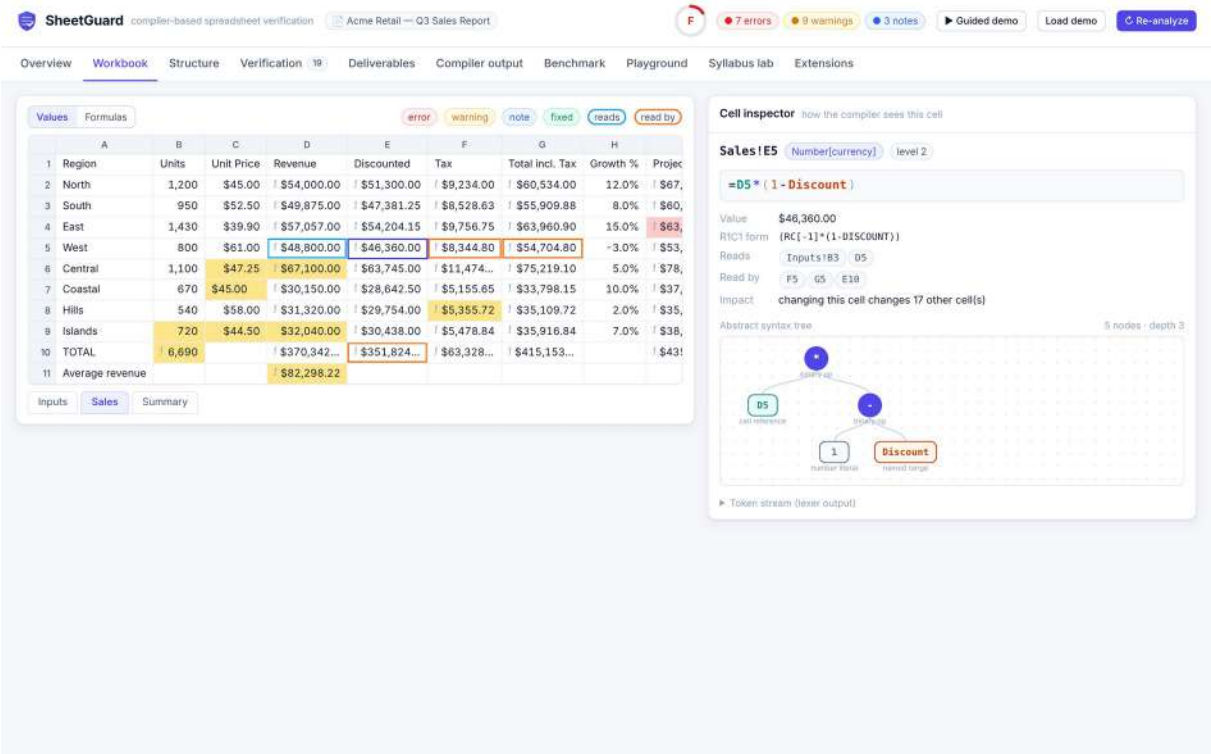


Figure 4.2: Workbook view: cells coloured by severity; the cell inspector shows the type Number[currency], R1C1 form, precedents, dependents, impact and AST of Sales!E5

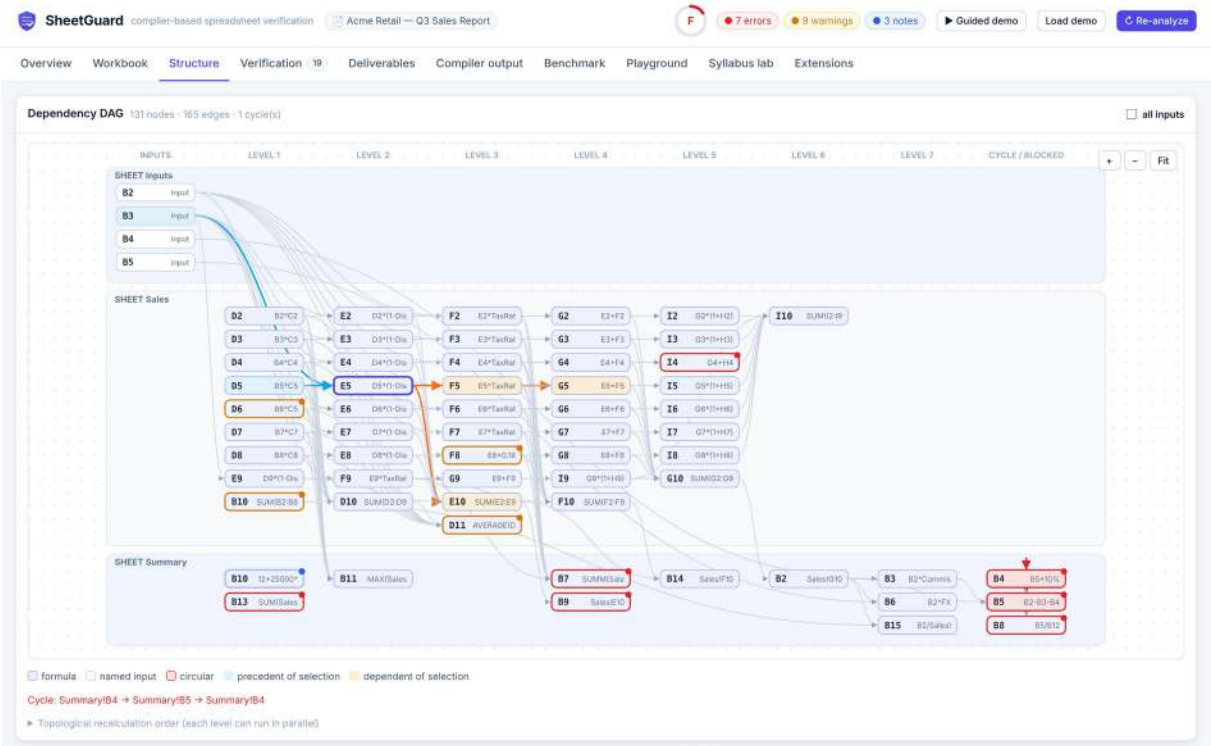


Figure 4.3: Structure: the dependency DAG (131 nodes, 165 edges) arranged by topological level; the Summary!B4 ↔ B5 cycle is set apart; precedents of the selection are blue, dependents orange

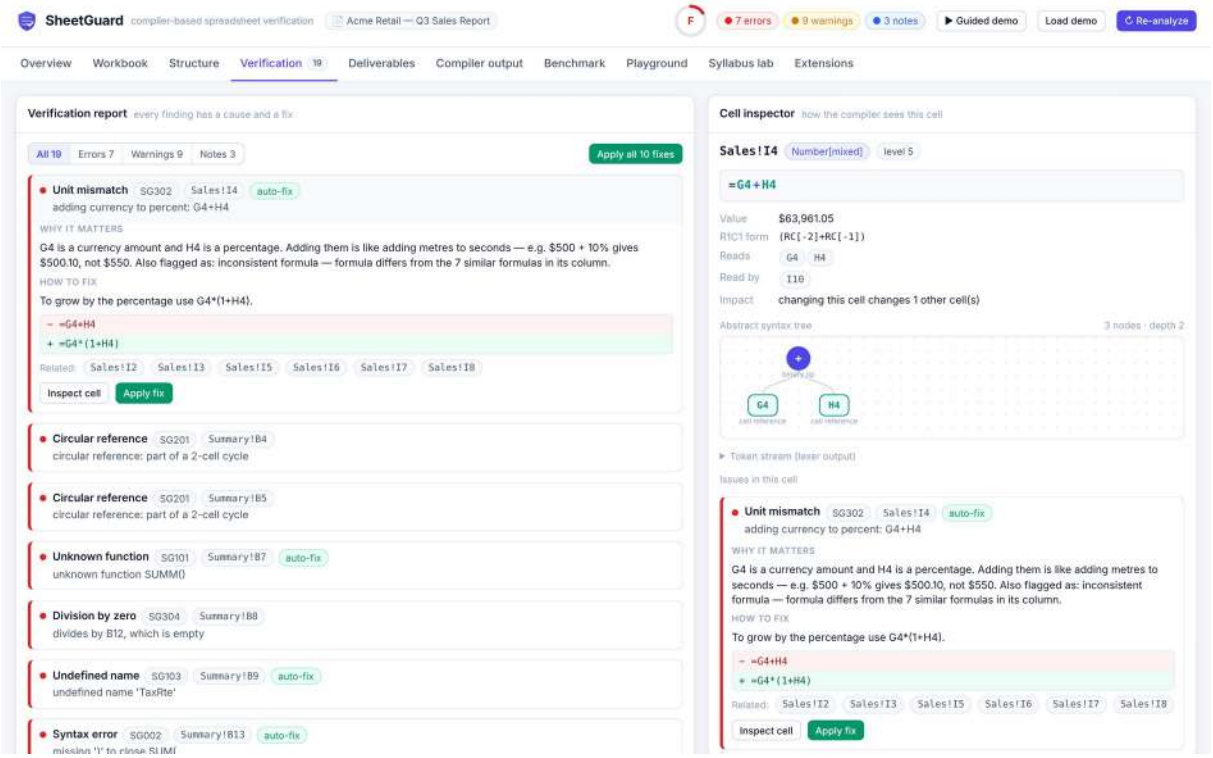


Figure 4.4: Verification report: 19 findings with codes, cells and auto-fix badges; an expanded finding shows why it matters and the corrected formula

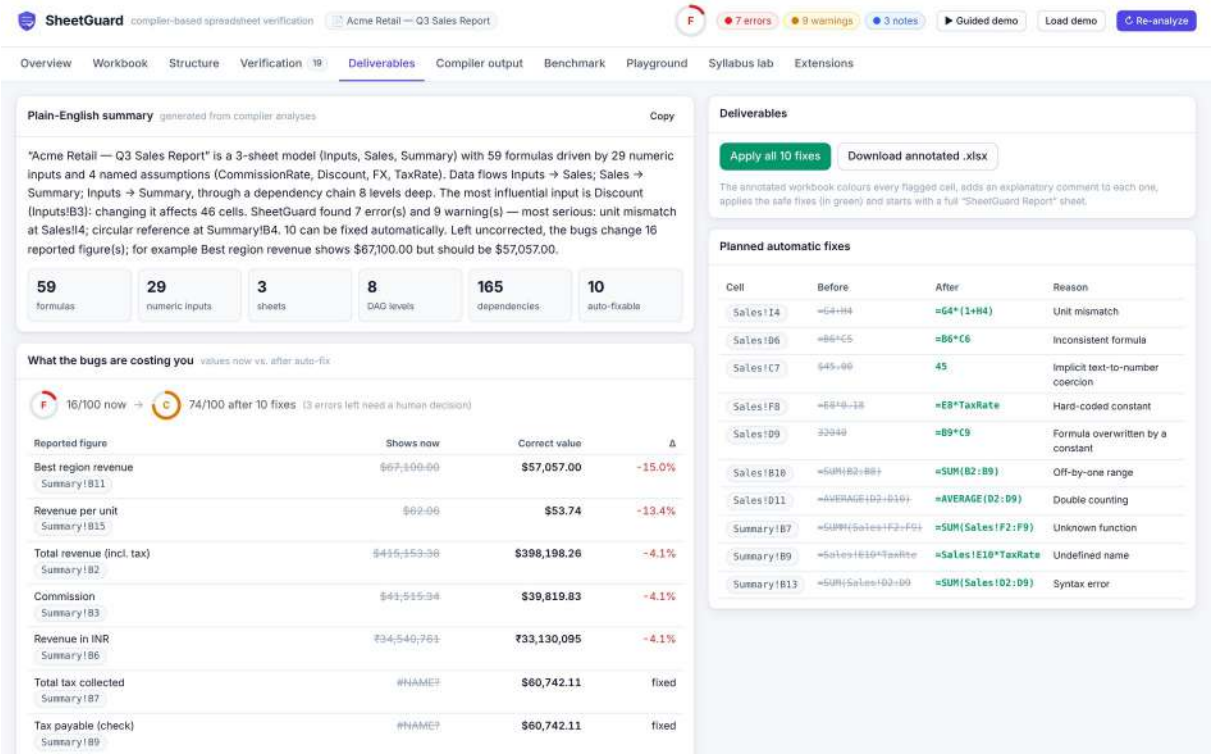


Figure 4.5: Deliverables: summary, health score, planned automatic fixes and download of the corrected, annotated workbook

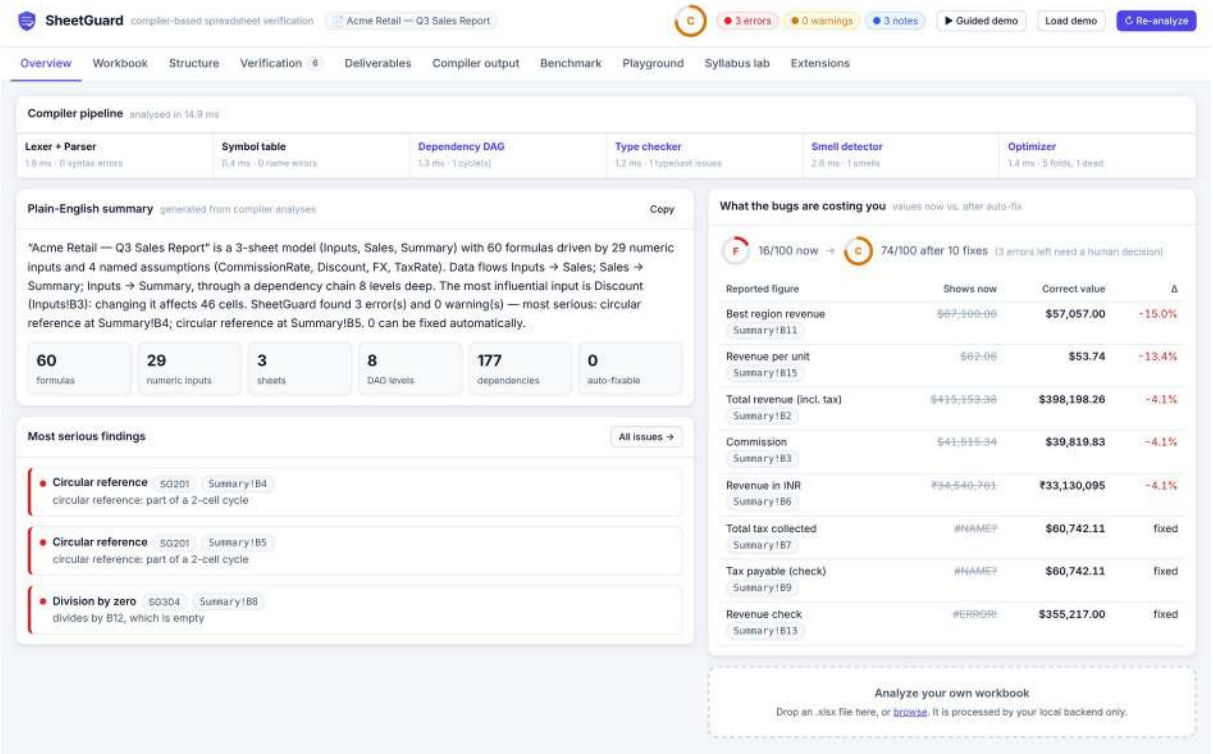


Figure 4.6: After applying all fixes: the health score rises from 16 (F) to 74 (C); three errors remain that need a human decision

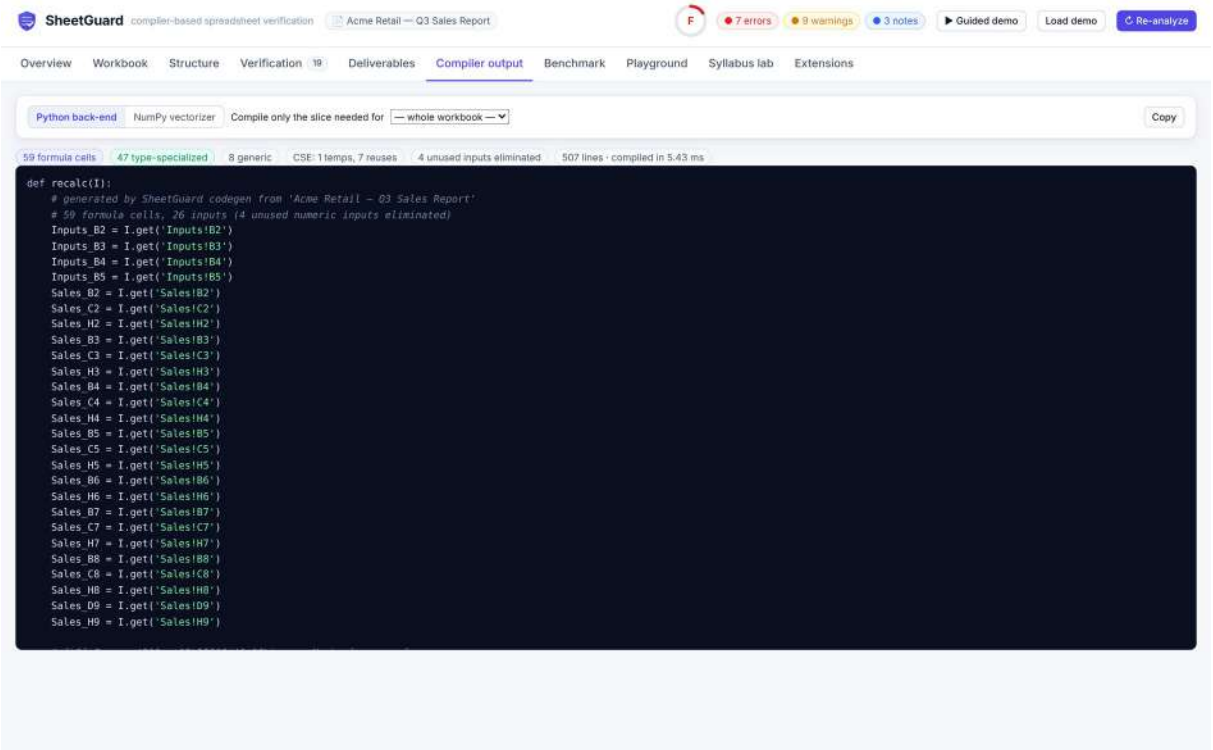


Figure 4.7: Compiler output: the straight-line Python generated for the whole workbook, with specialised fast paths and common subexpressions

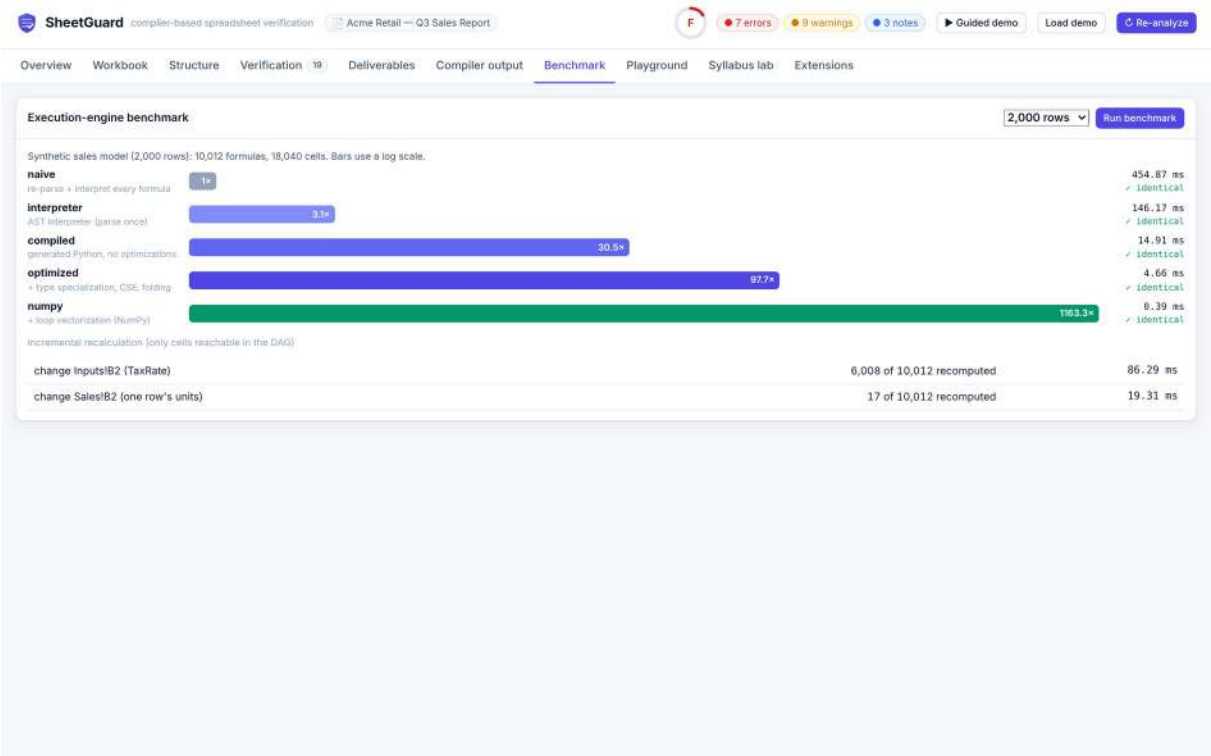


Figure 4.8: Benchmark view: naive interpretation vs. interpreter vs. compiled vs. optimised vs. NumPy, all producing identical results

4.2.2 Successful compilation and error detection

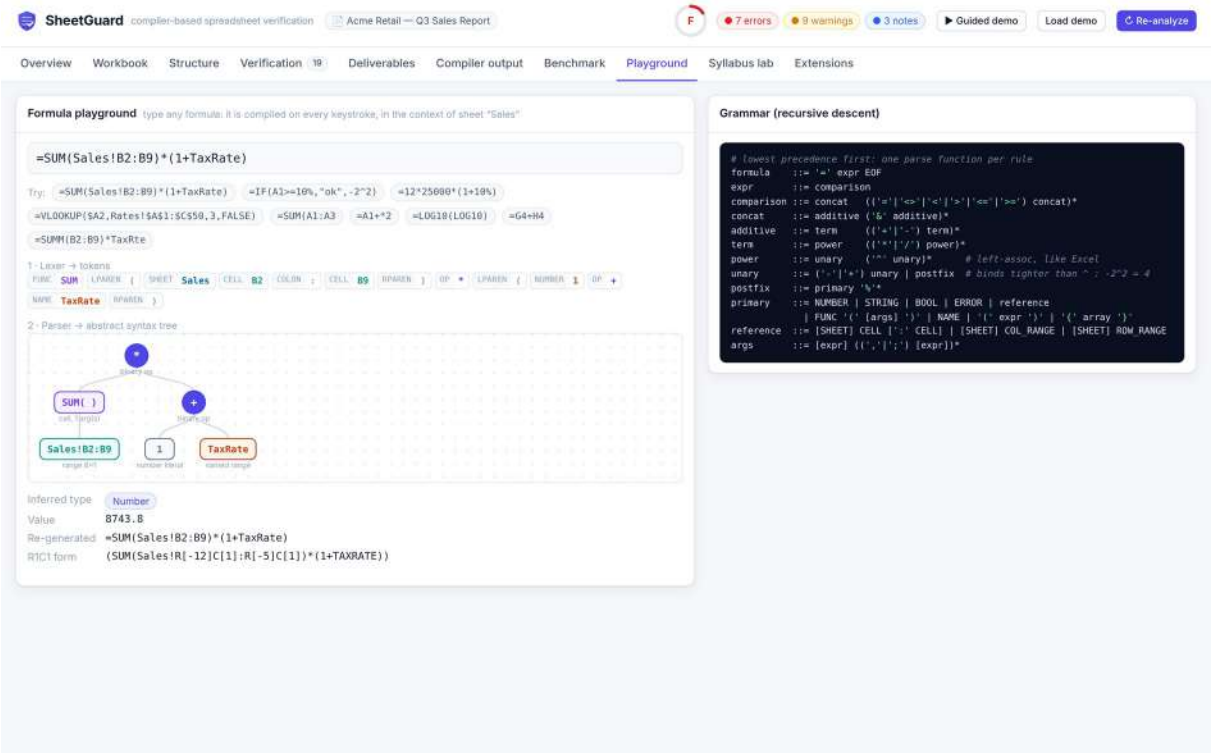


Figure 4.9: Playground: a formula compiled on every keystroke, showing tokens, the AST, inferred type, value, regenerated text and R1C1 form

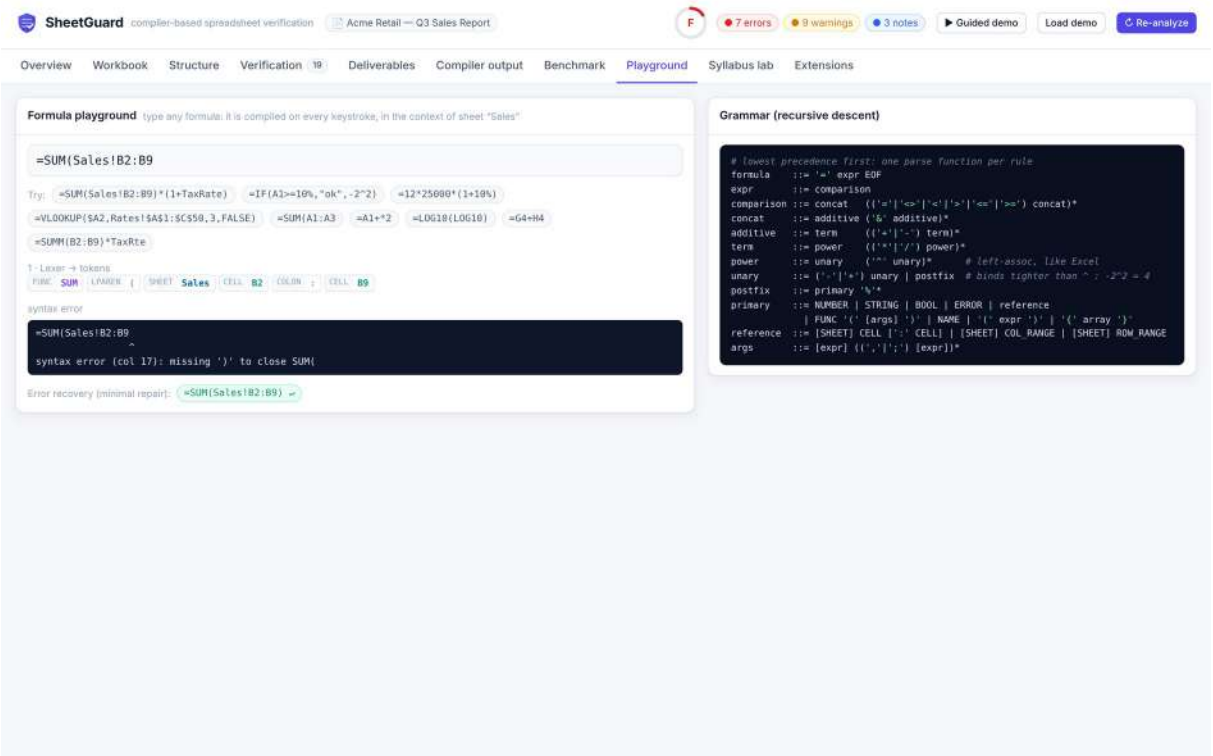


Figure 4.10: Syntax error detection: a missing ')' is reported with its position and a repaired formula is suggested

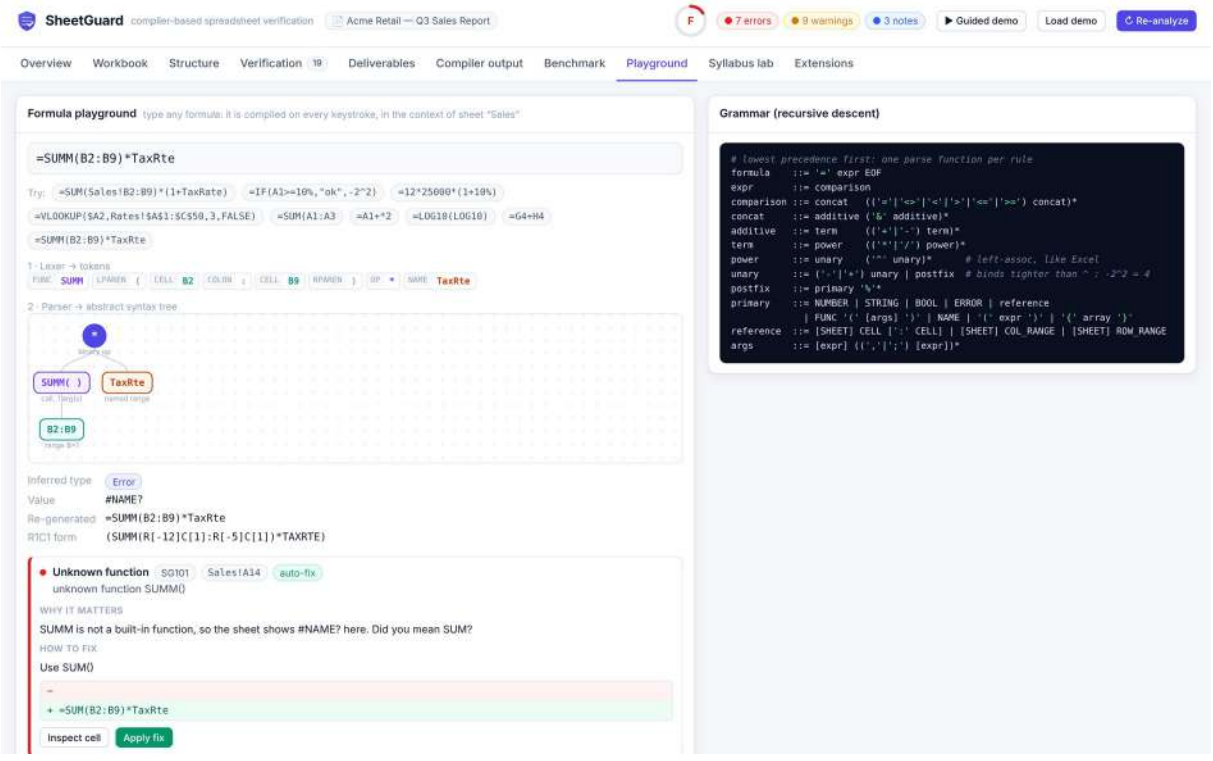


Figure 4.11: Semantic error detection: an unknown function (SUMM) and an undefined name (TaxRte), with suggestions

4.2.3 Syllabus lab

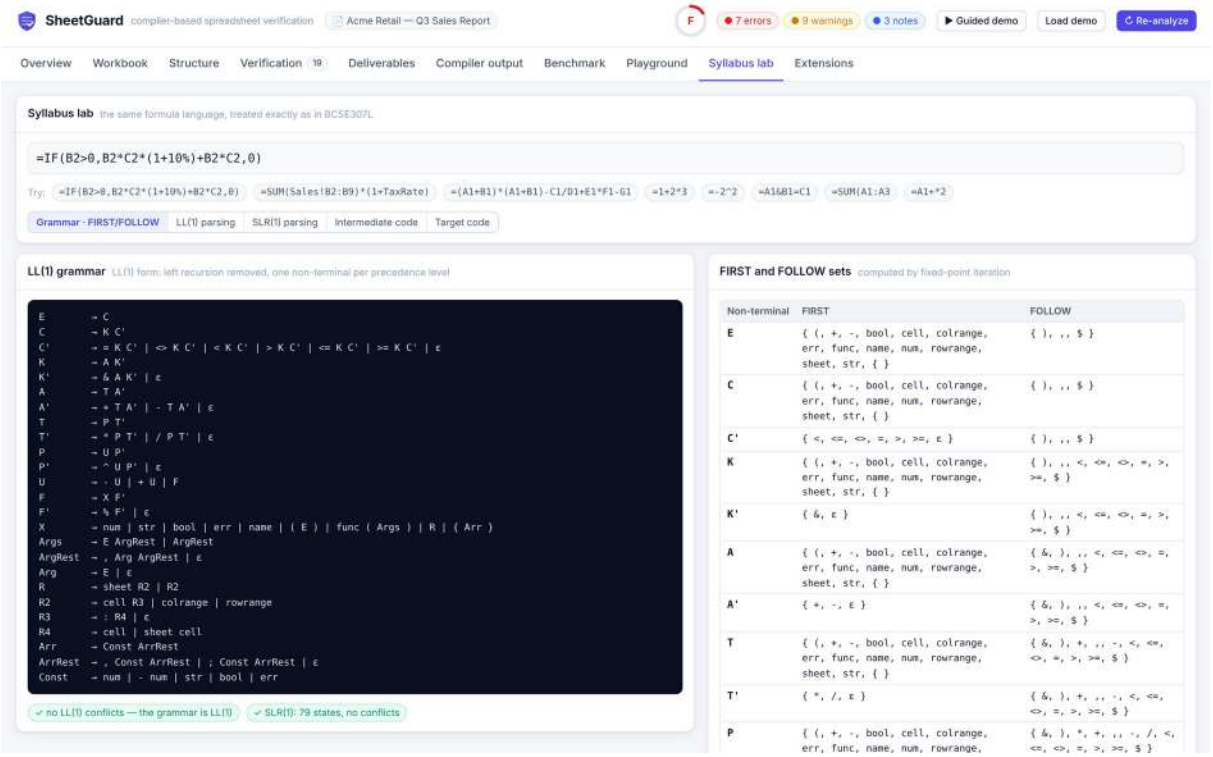


Figure 4.12: LL(1) grammar (left recursion removed) with FIRST and FOLLOW sets computed by fixed-point iteration

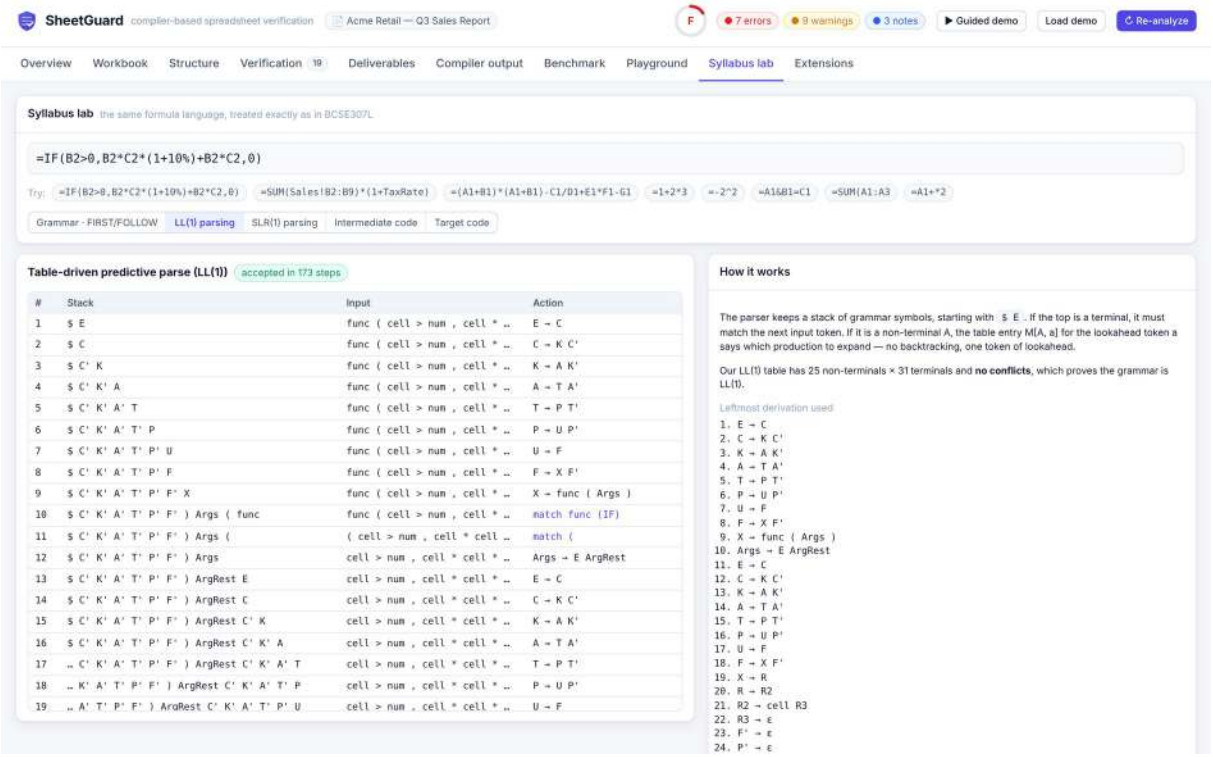


Figure 4.13: Table-driven LL(1) predictive parse of =IF(B2>0,B2*C2*(1+10%)+B2*C2,0) with stack, input and action at every step

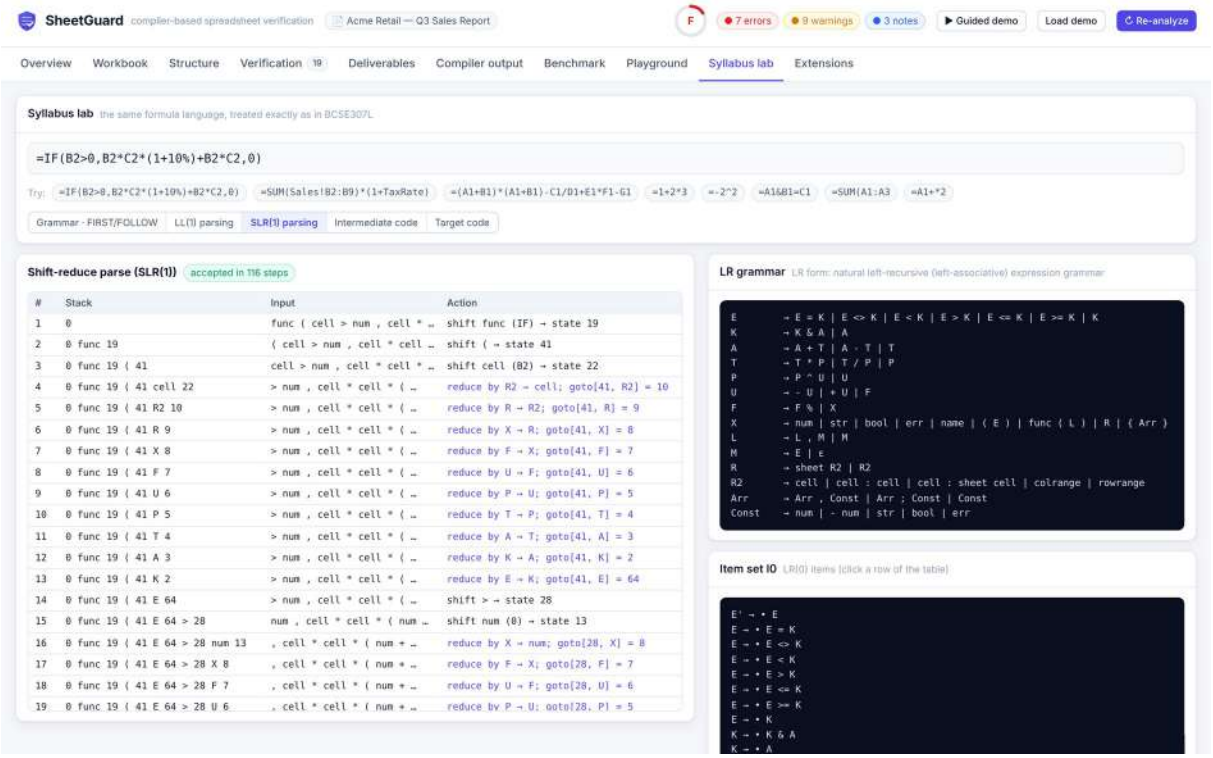


Figure 4.14: SLR(1) shift-reduce parse of the same formula, with the LR grammar and LR(0) item sets

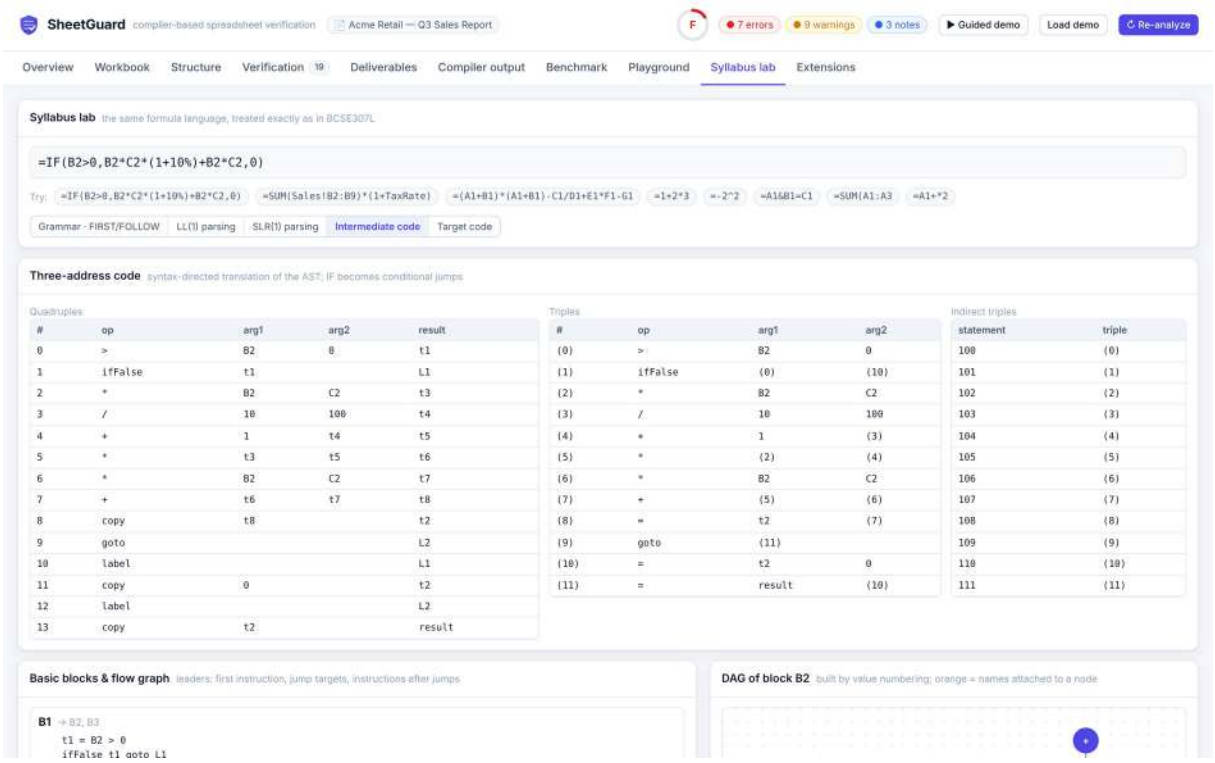


Figure 4.15: Three-address code as quadruples, triples and indirect triples; basic blocks, flow graph and DAG of a block

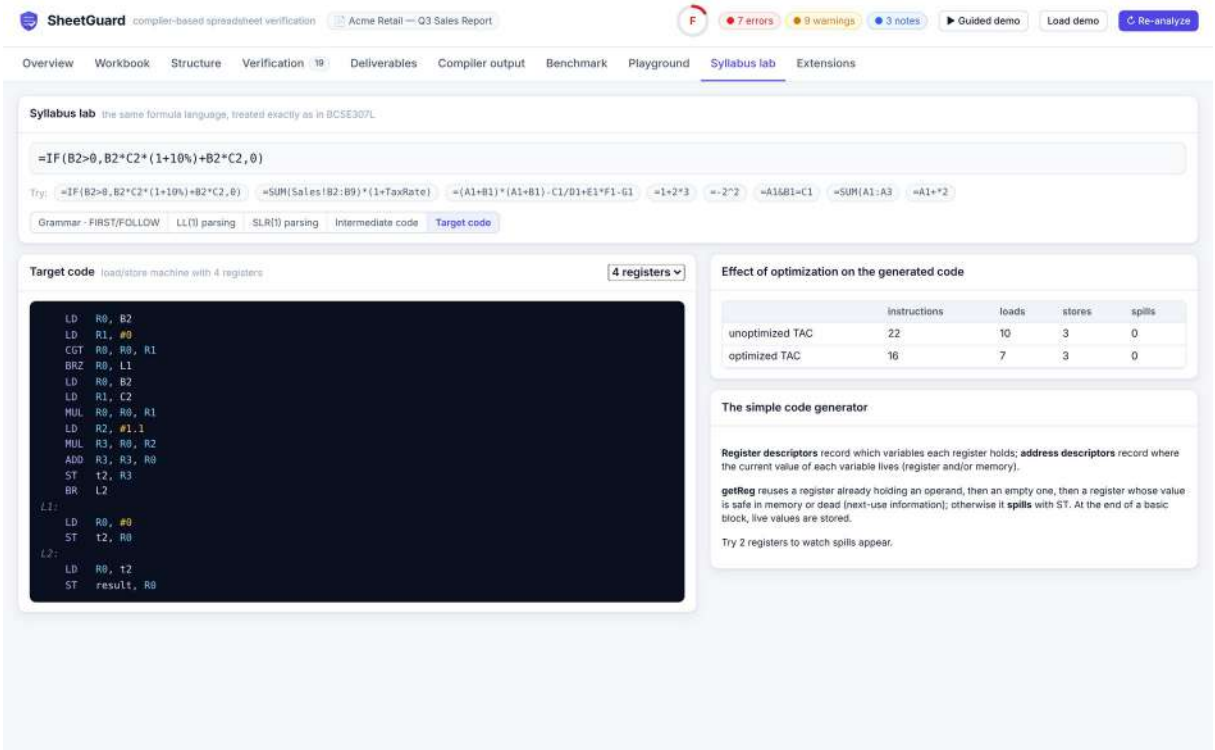


Figure 4.16: Target code for a register machine, and the effect of optimisation on instruction and register counts

4.2.4 Extensions

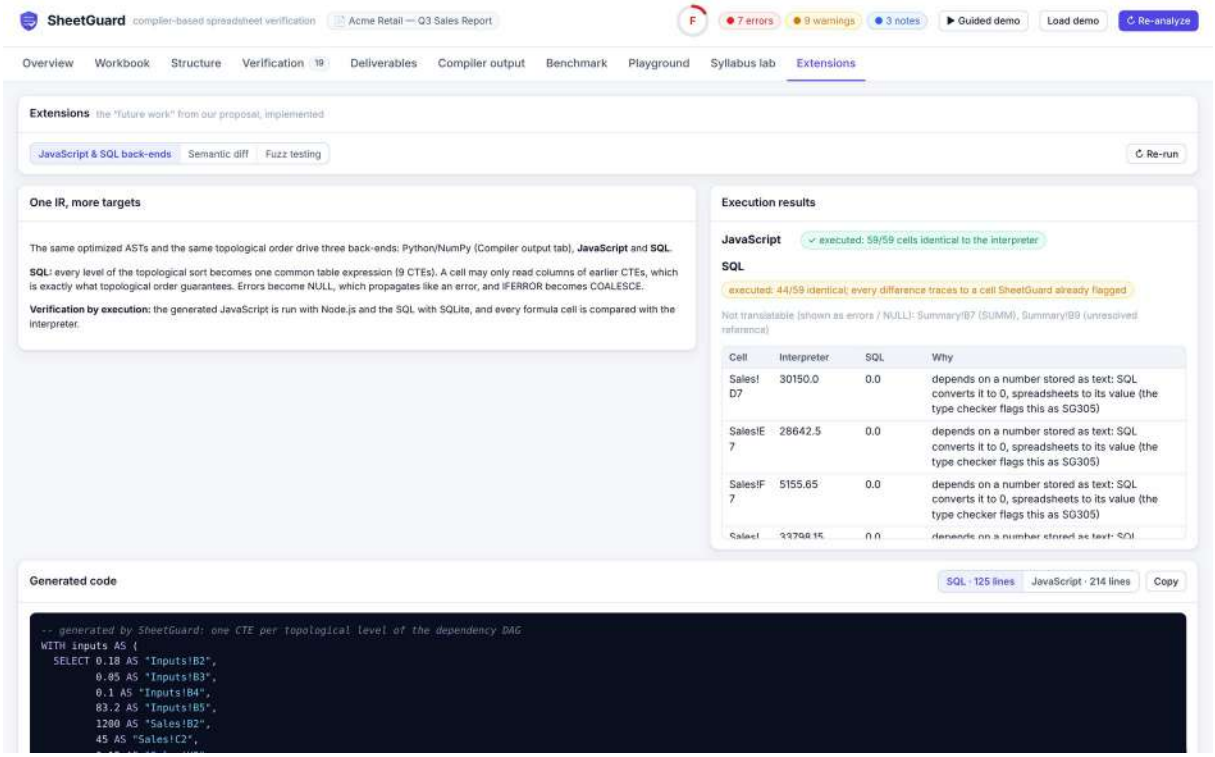


Figure 4.17: JavaScript and SQL back-ends, verified by execution against the interpreter

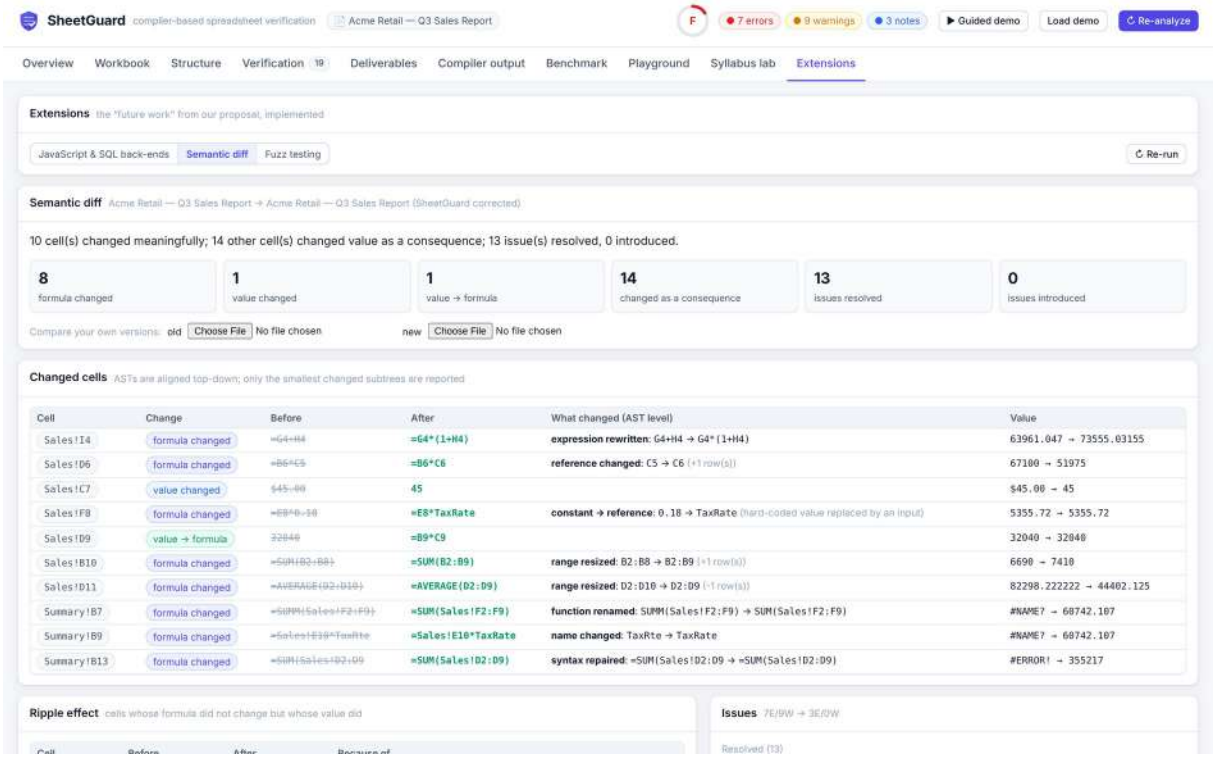


Figure 4.18: Semantic diff between the buggy and the auto-fixed workbook: minimal changed subtrees, ripple effect, issues resolved

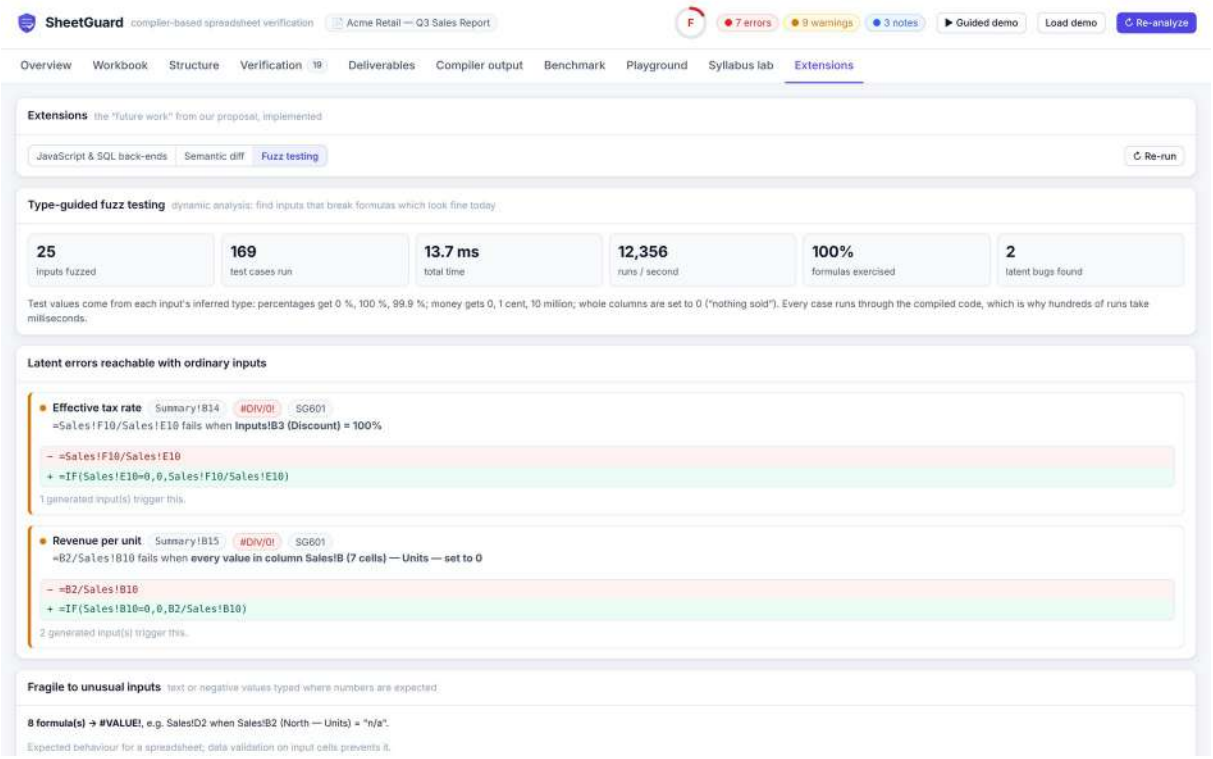


Figure 4.19: Type-guided fuzz testing: latent #DIV/0! errors with witness inputs and suggested guards

4.2.5 Google Sheets add-on and command line

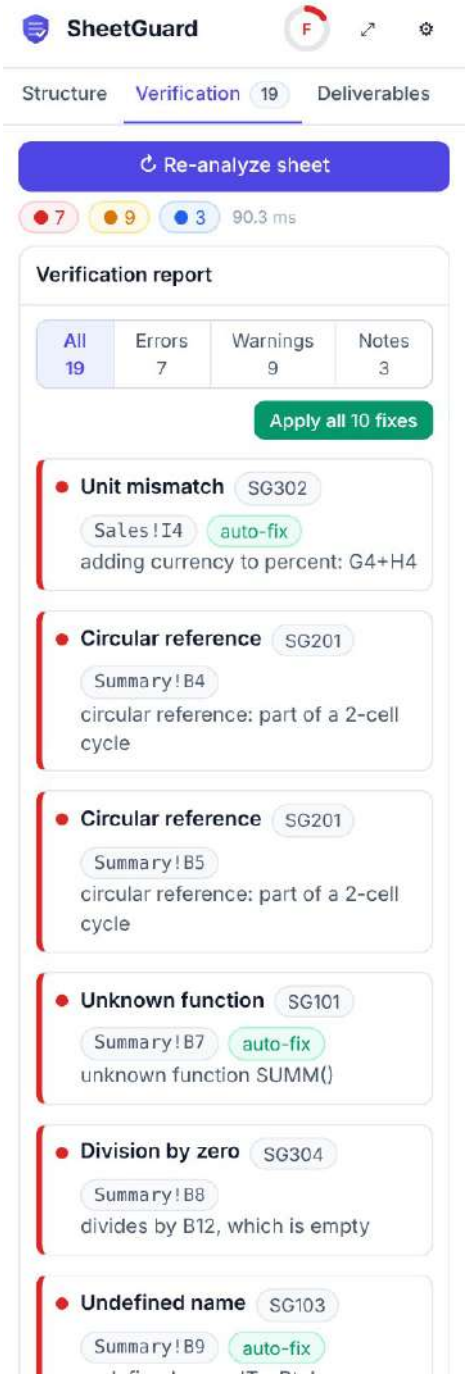


Figure 4.20: Google Sheets sidebar interface (300 px wide, as fixed by Google Sheets) after analysing the demo workbook

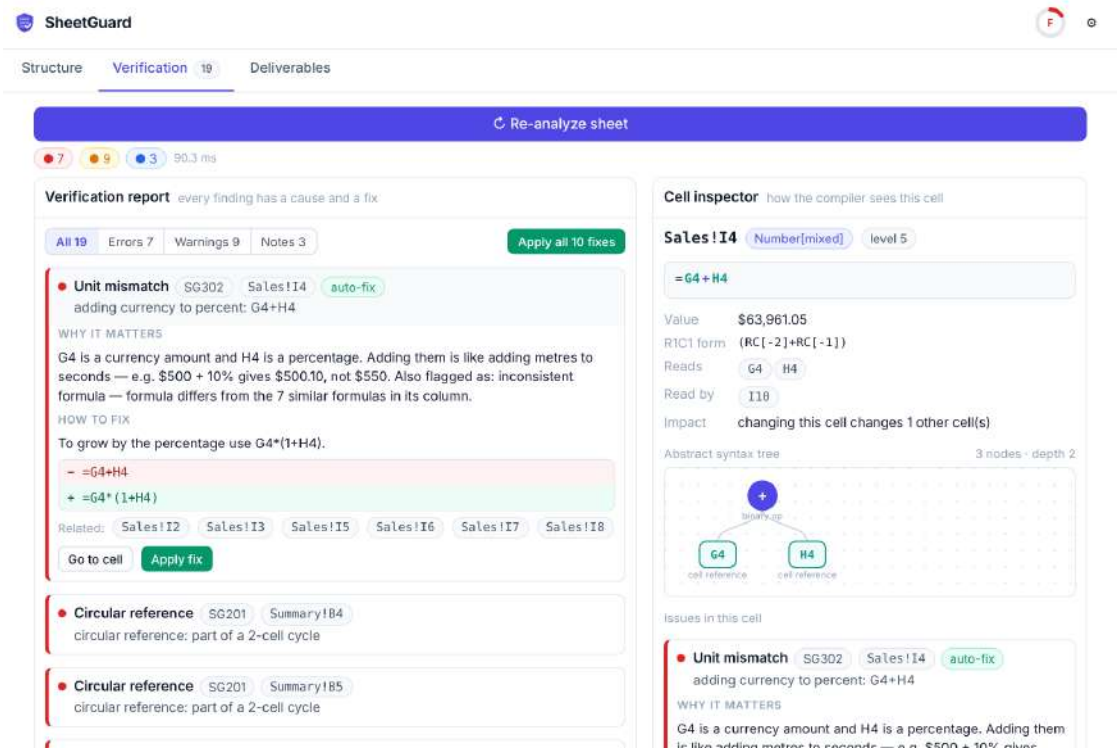


Figure 4.21: The add-on's large view: the same interface in a 1100 px dialog over the sheet, with the issue list beside the cell inspector

```
$ python -m sheetguard lex "=SUM(Sales!B2:B9)*(1+TaxRate)"
#   TYPE    LEXEME   POS
--  ------  -------  -----
0   FUNC    SUM      1-4
1   LPAREN  (        4-5
2   SHEET   Sales    5-11
3   CELL    B2       11-13
4   COLON   :        13-14
5   CELL    B9       14-16
6   RPAREN  )        16-17
7   OP      *        17-18
8   LPAREN  (        18-19
9   NUMBER  1        19-20
10  OP      +        20-21
11  NAME    TaxRate  21-28
12  RPAREN  )        28-29
13  EOF     <EOF>    29-29

$ python -m sheetguard parse "=SUM(Sales!B2:B9)*(1+TaxRate)"
BinOp *
├─ Call SUM/1
│   └─ Range Sales!B2:B9  [8×1]
└─ BinOp +
    ├─ Number 1
    └─ Name TaxRate

re-generated: =SUM(Sales!B2:B9)*(1+TaxRate)
```

Figure 4.22: Command line: token stream and abstract syntax tree of a formula

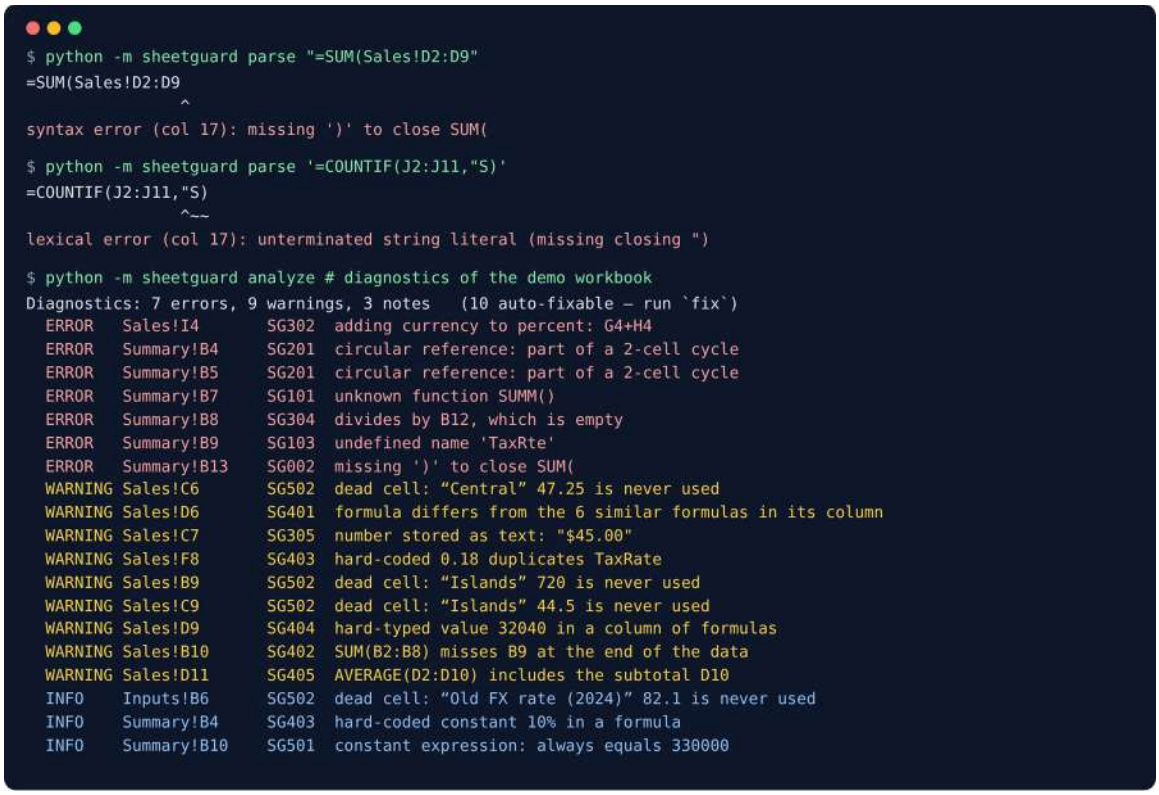


Figure 4.23: Command line error detection: caret-positioned syntax and lexical errors, and the diagnostics of the demo workbook (7 errors, 9 warnings, 3 notes)



Figure 4.24: Command line: 10 verified automatic fixes and the corrected, annotated .xlsx



Figure 4.25: JavaScript back-end verified on all 59 formulas; SQL differences are explained (SQL converts text to 0); fuzzing finds two latent bugs

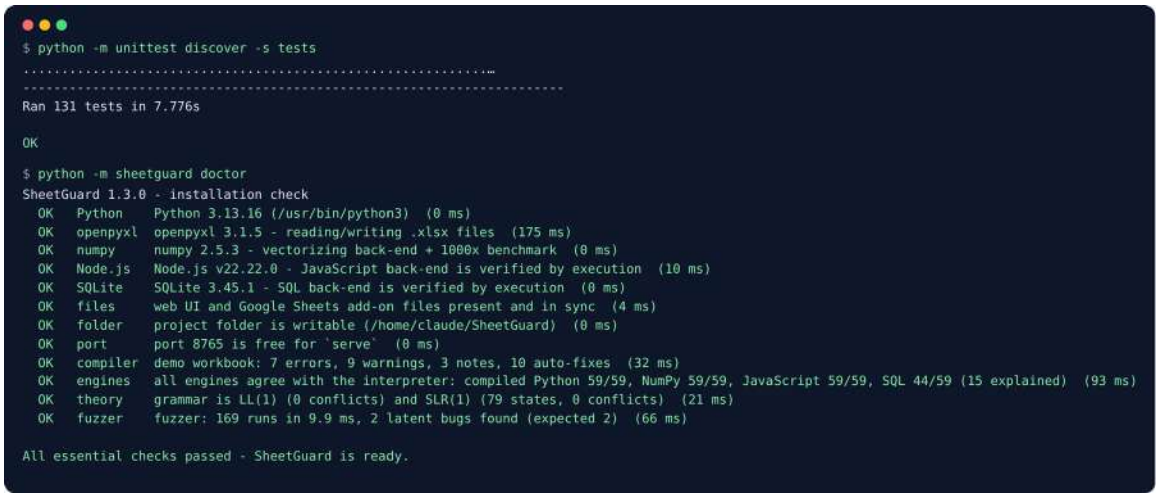


Figure 4.26: All 131 automated tests pass, and the doctor self-test confirms that every engine agrees with the interpreter

4.3 Current Progress and Challenges

Completed modules. All planned modules are implemented and tested:

Table 4.3: Status of the modules

Module	Status
Lexer, parser, AST, caret-positioned errors, phrase-level recovery	Completed
Symbol table, dependency DAG, cycle detection, topological order, impact analysis	Completed
Type, unit and dimension checker; smell detector; verified auto-fix; annotated .xlsx export	Completed
Optimiser, interpreter, Python code generator, NumPy vectoriser, benchmark	Completed
HTTP back-end, web application, Google Sheets add-on (sidebar, large view, graph dialog)	Completed
Plain-English summary, bug-impact report, health score, guided demo	Completed
Syllabus lab: FIRST/FOLLOW, LL(1), SLR(1), TAC, basic blocks, DAG, target code	Completed

Module	Status
Extensions: JavaScript and SQL back-ends, semantic diff, fuzz testing	Completed
Hardening: differential testing, malformed and very large inputs, installation self-test	Completed
Second test workbook (semester grade book, 17 planted bugs)	Completed

Pending work.

- Publishing the add-on on the Google Workspace Marketplace, which requires Google's app verification; it currently runs as a script bound to one spreadsheet.
- Permanent hosting of the back-end; it currently runs on the user's computer and is reached from Google Sheets through a temporary tunnel URL.
- Executing array and dynamic-array functions (for example `FILTER`, `QUERY`): 213 functions are recognised but 129 are executed; for the rest the cached value is used and note SG106 explains this.

Challenges encountered and how they were solved.

- **Exact spreadsheet semantics.** Differential testing of five engines exposed many subtle rules: `SUM` over a reference skips text while `SUM` of a text literal is an error, `COUNT` ignores errors, numbers convert to text with 15 significant digits, and a call with the wrong number of arguments gives `#N/A`. Each rule was implemented once in the shared runtime and pinned by a regression test.
- **Apps Script rewrote the sidebar code.** In the first real Google Sheets run the sidebar was blank. An in-page error reporter showed “Invalid or unexpected token”: HtmlService re-serialises inline scripts before writing them into its sandbox frame and corrupted the 120 KB script. The build now embeds the script base64-encoded and decodes it in the browser, so it cannot be rewritten.
- **Typed values from Google Sheets.** The JSON loader converted the text `"78"` into the number 78, which hid “number stored as text” errors coming from Google Sheets. JSON values are now kept exactly as typed.
- **Deep and large inputs.** Formulas nested 100 levels deep and chains of 10,000 cells exceeded Python's recursion limit; analysis now runs in threads with a large stack. Whole-column ranges with a stray value far below the data were clipped per column to keep analysis linear.
- **Installation on Windows.** Extracting the zip into an existing folder merged `SheetGuard` and `sheetguard` (Windows paths are case-insensitive) and broke the package; a pre-release Python version was also picked up. The archive's top folder was renamed, an install check now detects a mixed-up folder, and the launcher prefers stable Python versions.

4.4 Conclusion and Future Enhancements

SheetGuard shows that the techniques of a compiler design course apply directly to the most widely used programming language in the world. Treating a workbook as a program, it lexes and parses every formula with a grammar proven LL(1) and SLR(1), resolves names through nested scopes, builds a dependency graph on which cycles, evaluation order and impact are computed, infers types with units, detects structural smells with the R1C1 canonical form, optimises the formulas and compiles them to five agreeing execution engines. On two test workbooks it finds all 31 planted bugs, explains each in plain English, and repairs most of them with fixes that are re-verified before they are offered; on the demonstration report this corrects a figure that was overstated by 15%. Compilation and vectorisation make evaluation about 720 times faster than naive interpretation. The work is delivered where spreadsheet users already work, as a Google Sheets add-on, together with a web application and a command-line tool, and is backed by 131 automated tests and extensive differential testing.

Future enhancements.

- Publish the add-on on the Google Workspace Marketplace with a permanently hosted back-end.
- Execute array and dynamic-array functions (`FILTER`, `SORT`, `UNIQUE`, `QUERY`) and `LAMBDA`, completing the runtime.
- Learn units and intent from row and column headers, as in UCheck [6], in addition to number formats.
- Analyse a sheet incrementally as the user types, reusing the incremental recalculation machinery, and offer an Excel add-in using the same back-end.
- Evaluate the detectors on public corpora of real spreadsheets to measure precision and recall.

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.

[2] R. R. Panko, "What we know about spreadsheet errors," *Journal of End User Computing*, vol. 10, no. 2, pp. 15–21, 1998 (revised 2008).

[3] T. Herndon, M. Ash and R. Pollin, "Does high public debt consistently stifle economic growth? A critique of Reinhart and Rogoff," *Cambridge Journal of Economics*, vol. 38, no. 2, pp. 257–279, 2014.

[4] JPMorgan Chase & Co., "Report of JPMorgan Chase & Co. Management Task Force regarding 2012 CIO losses," Jan. 2013.

[5] M. Erwig and M. Burnett, "Adding apples and oranges," in *Proc. 4th Int. Symp. Practical Aspects of Declarative Languages (PADL)*, LNCS 2257, Springer, 2002, pp. 173–191.

[6] R. Abraham and M. Erwig, "UCheck: A spreadsheet type checker for end users," *Journal of Visual Languages and Computing*, vol. 18, no. 1, pp. 71–95, 2007.

[7] F. Hermans, M. Pinzger and A. van Deursen, "Detecting code smells in spreadsheet formulas," in *Proc. 28th IEEE Int. Conf. Software Maintenance (ICSM)*, 2012, pp. 409–418.

[8] S.-C. Cheung, W. Chen, Y. Liu and C. Xu, "CUSTODES: Automatic spreadsheet cell clustering and smell detection using strong and weak features," in *Proc. 38th Int. Conf. Software Engineering (ICSE)*, 2016, pp. 464–475.

[9] D. W. Barowy, E. D. Berger and B. Zorn, "ExceLint: Automatically finding spreadsheet formula errors," *Proc. ACM on Programming Languages*, vol. 2, no. OOPSLA, Art. 148, 2018.

[10] D. W. Barowy, D. Gochev and E. D. Berger, "CheckCell: Data debugging for spreadsheets," in *Proc. ACM OOPSLA*, 2014, pp. 507–523.

[11] P. Sestoft, *Spreadsheet Implementation Technology: Basics and Extensions*. Cambridge, MA, USA: MIT Press, 2014.

[12] S. Peyton Jones, A. Blackwell and M. Burnett, "A user-centred approach to functions in Excel," in *Proc. 8th ACM SIGPLAN Int. Conf. Functional Programming (ICFP)*, 2003, pp. 165–176.

[13] R. E. Tarjan, "Depth-first search and linear graph algorithms," *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.

[14] A. B. Kahn, "Topological sorting of large networks," *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962.

[15] W. M. McKeeman, "Differential testing for software," *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.

[16] G. C. Necula, “Translation validation for an optimizing compiler,” in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2000, pp. 83–94.

[17] OASIS, “Open Document Format for Office Applications (OpenDocument) Version 1.2, Part 2: Recalculated Formula (OpenFormula) Format,” OASIS Standard, 2011.

[18] L. Kelion, “Excel: Why using Microsoft's tool caused Covid-19 results to be lost,” *BBC News*, 5 Oct. 2020.

[19] Google, “HTML Service: Create and serve HTML,” Google Apps Script developer documentation. [Online]. Available: https://developers.google.com/apps-script/guides/html